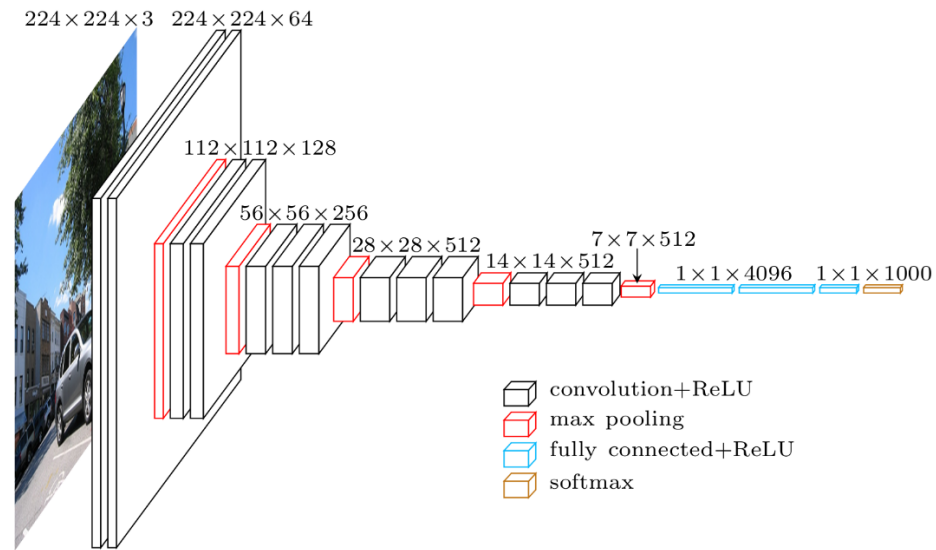




Popular CNNs

Most Popular CNN Architectures Clearly Explained and Visualized

LeNet (video)

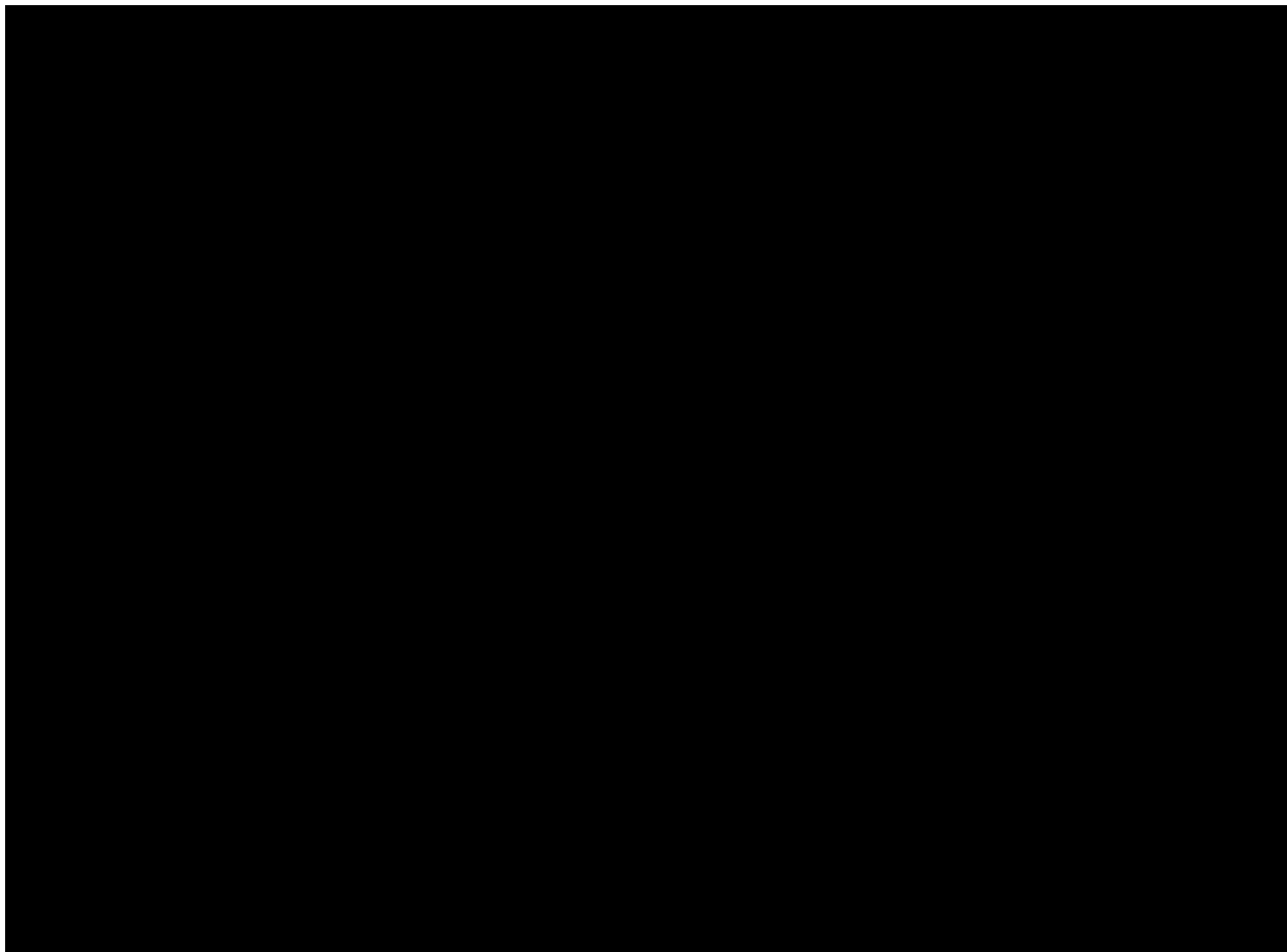


Image net Dataset

The **ImageNet** dataset contains 14,197,122 annotated images according to the WordNet hierarchy. Since 2010 the dataset is used in the ImageNet Large Scale Visual Recognition Challenge (ILSVRC), a benchmark in image classification and object detection.





Fei-Fei Li, Professor of Computer Science at Stanford University



mite



container ship



motor scooter



leopard

	mite
	black widow
	cockroach
	tick
	starfish

	container ship
	lifeboat
	amphibian
	fireboat
	drilling platform

	motor scooter
	go-kart
	moped
	bumper car
	golfcart

	leopard
	jaguar
	cheetah
	snow leopard
	Egyptian cat



grille



mushroom



cherry



Madagascar cat

	convertible
	grille
	pickup
	beach wagon
	fire engine

	agaric
	mushroom
	jelly fungus
	gill fungus
	dead-man's-fingers

	dalmatian
	grape
	elderberry
	ffordshire bullterrier
	currant

	squirrel monkey
	spider monkey
	titi
	indri
	howler monkey

Image net Dataset

Top-5 error

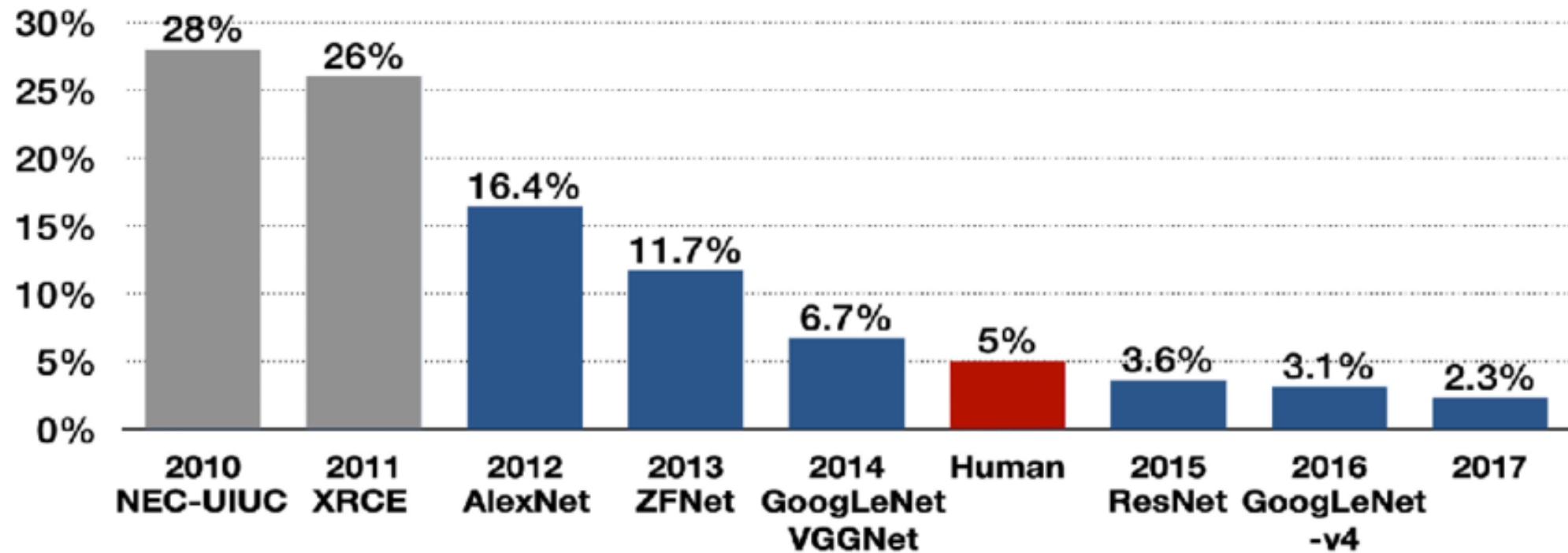


Image net Dataset

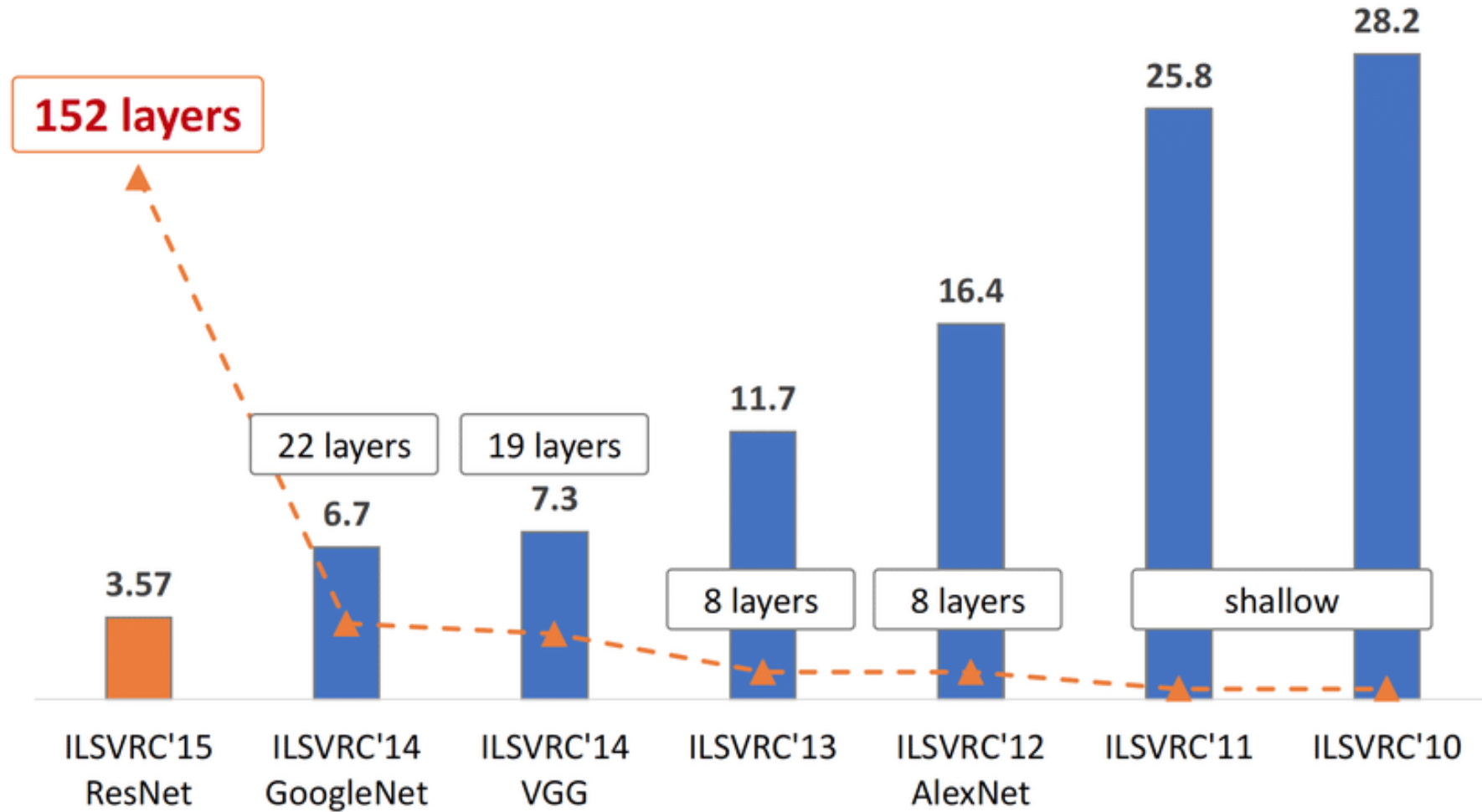


Image net Dataset

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners

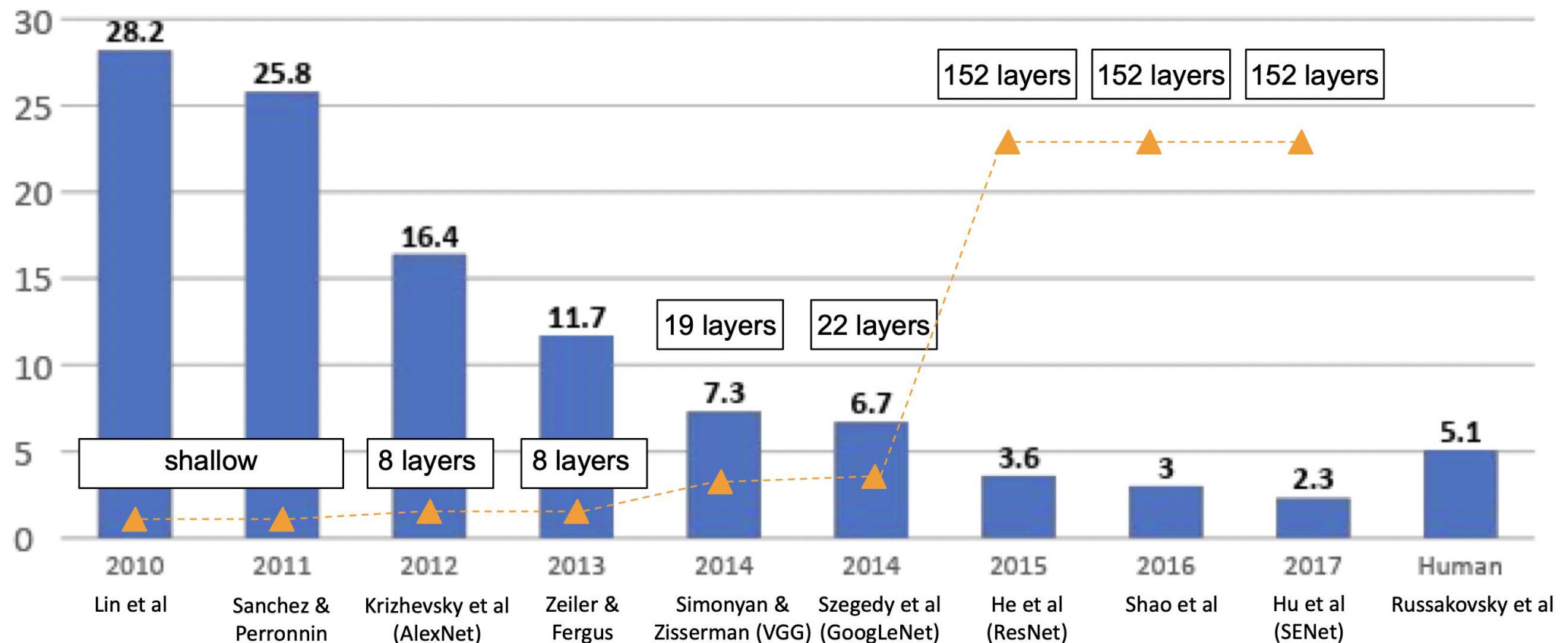


Image net Dataset

Image Classification on ImageNet

Leaderboard

Dataset

View

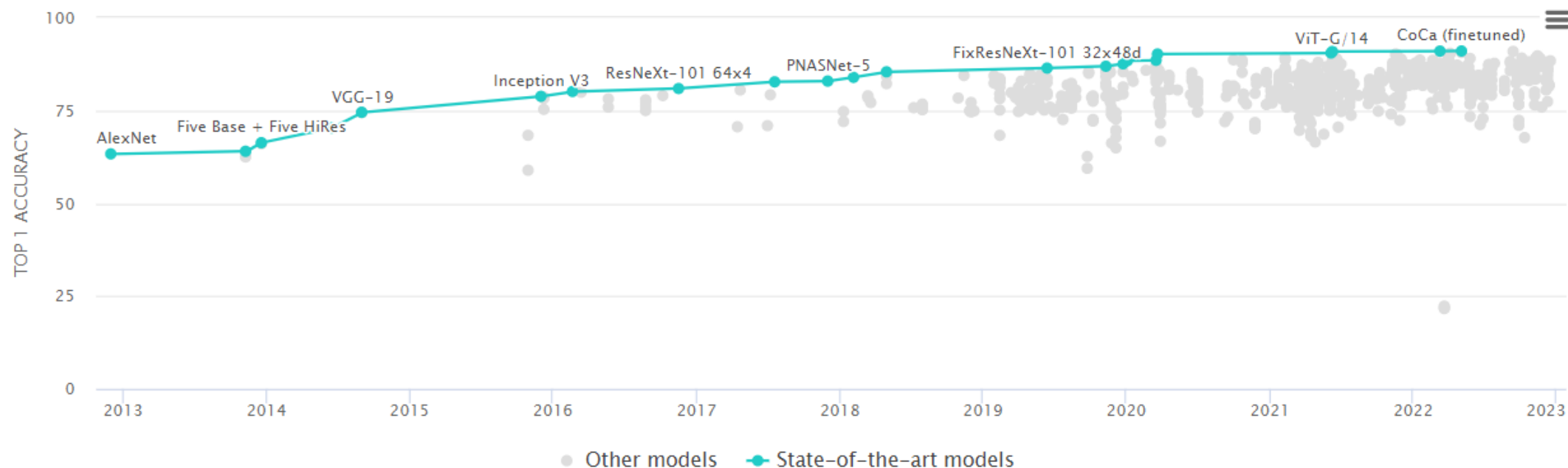
Top 1 Accuracy

by

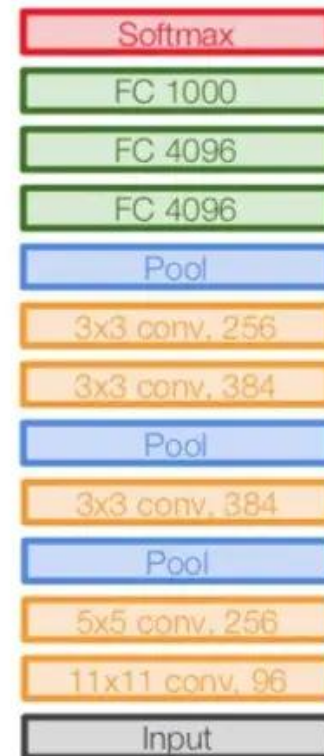
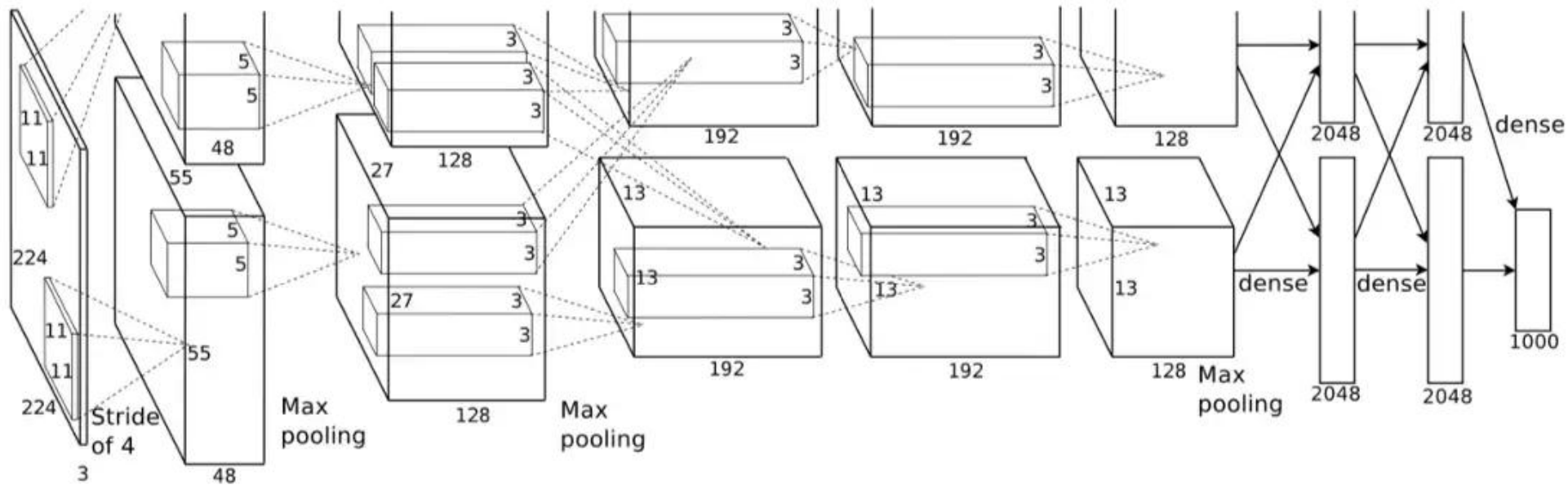
Date

for

All models

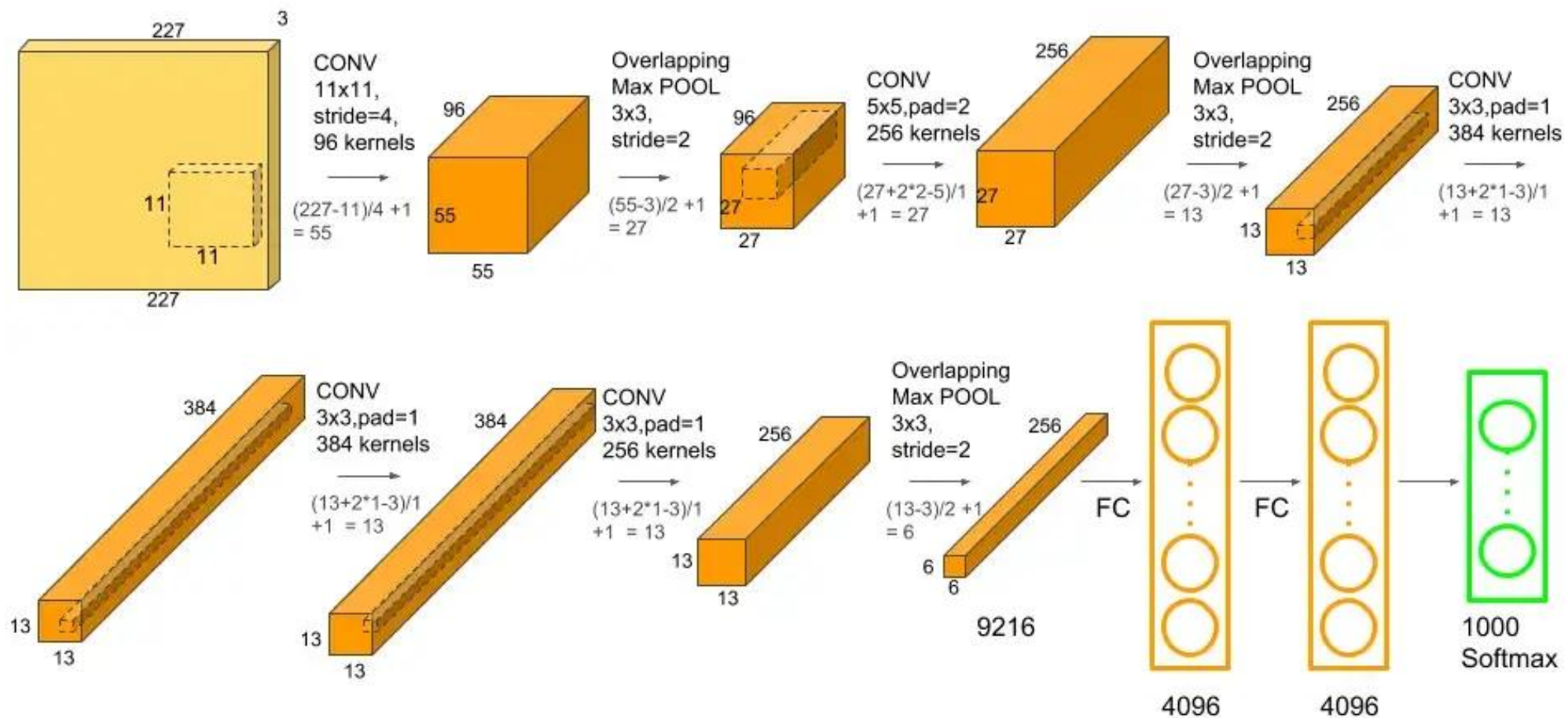


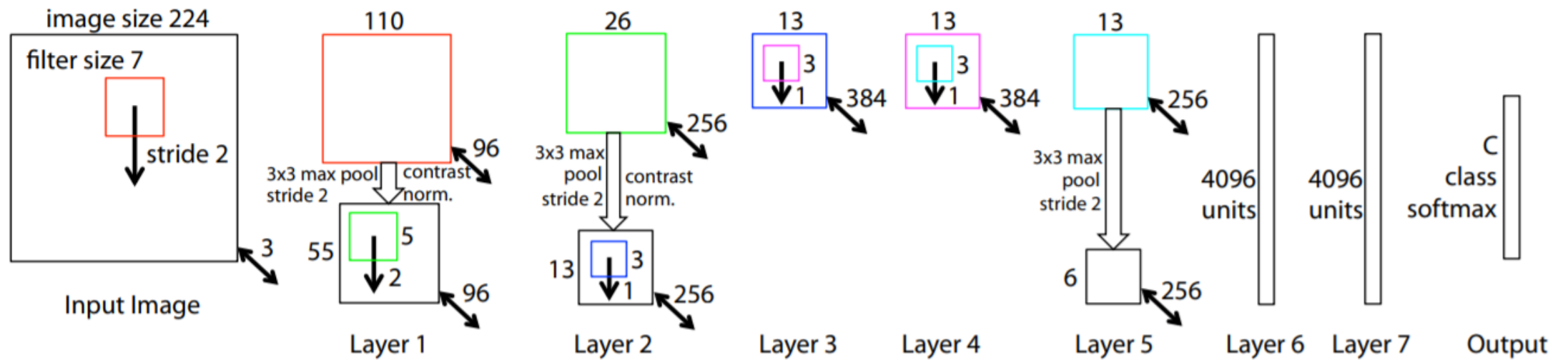
AlexNet



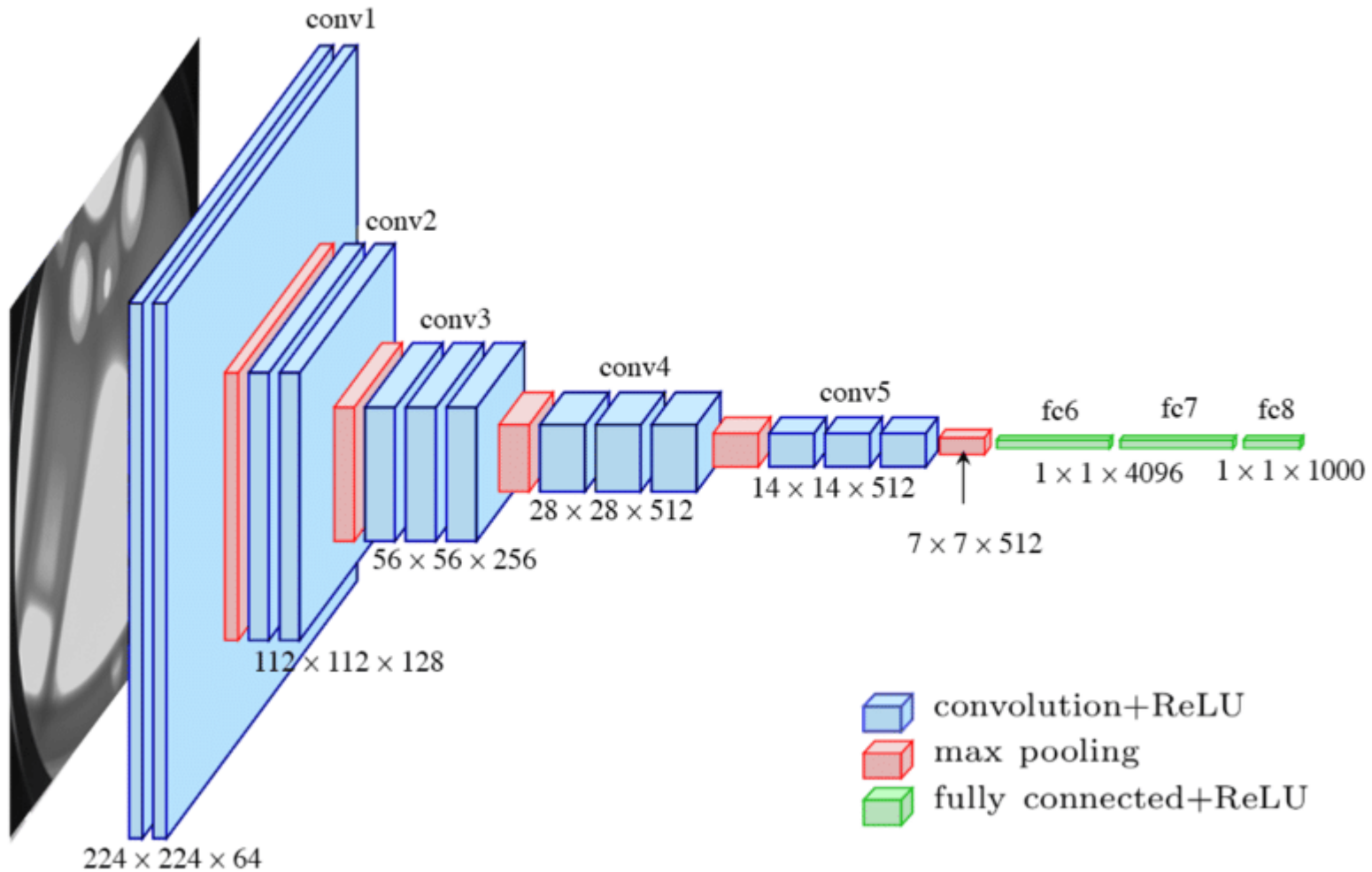
AlexNet

AlexNet

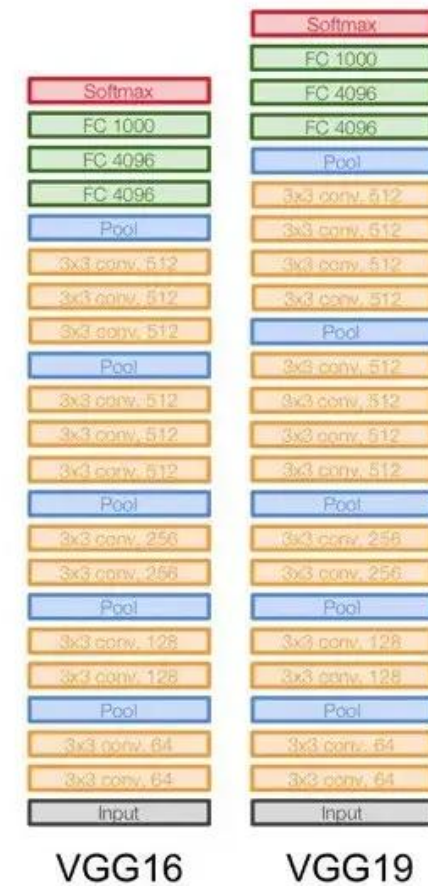
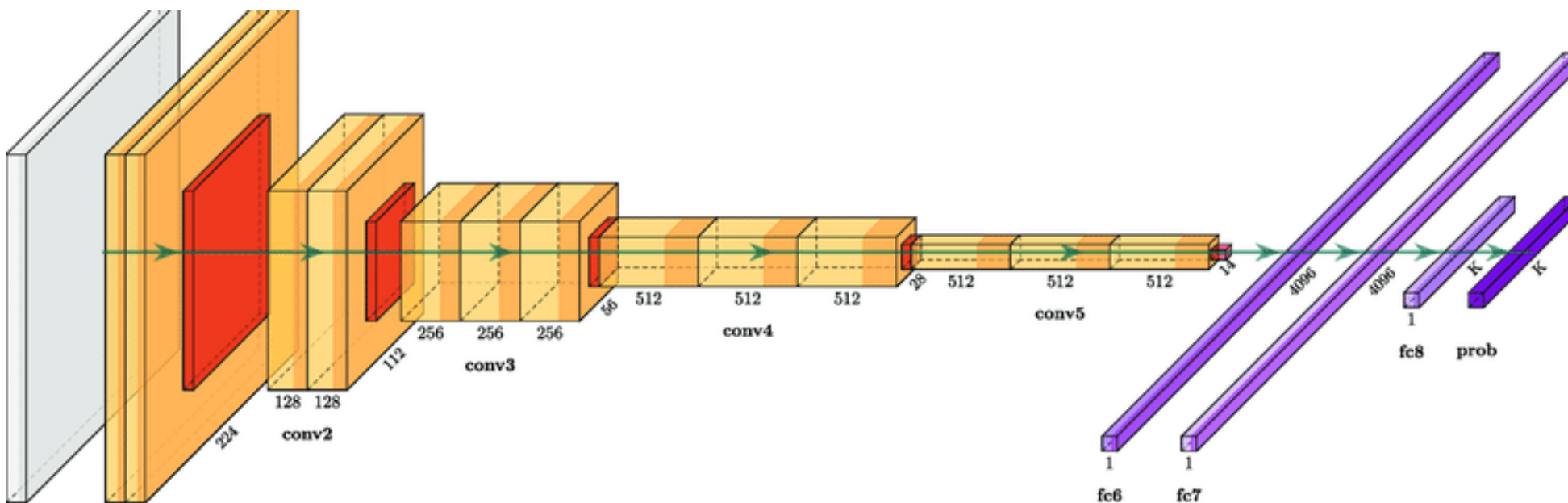




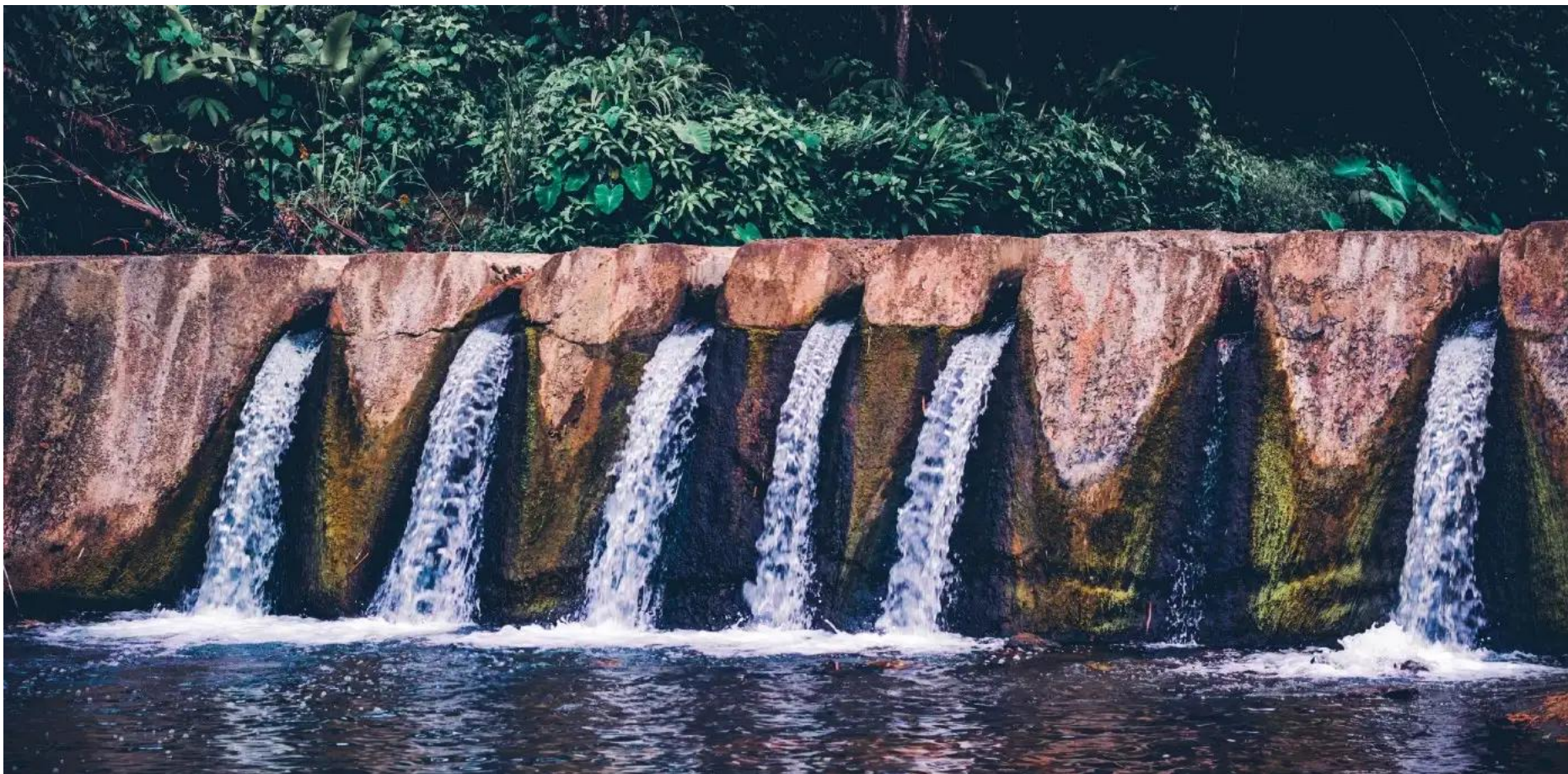
VGG Net



VGG Net



Batch Normalization



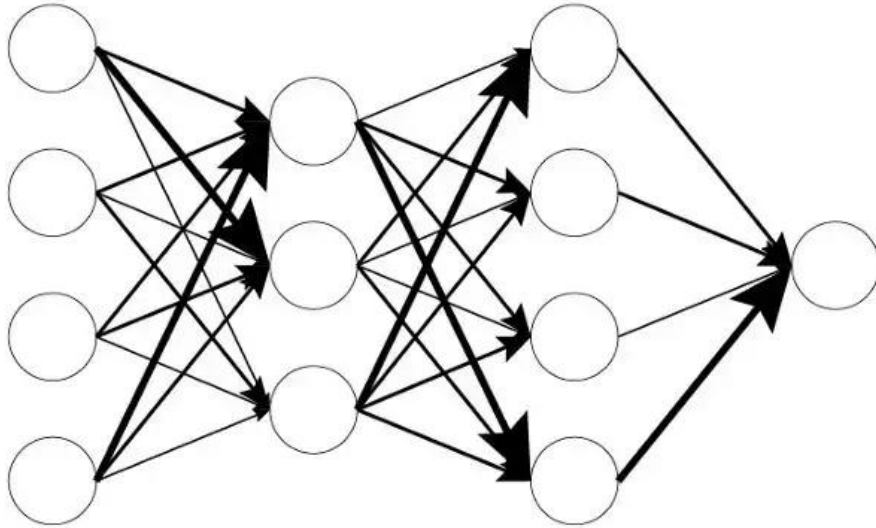
Batch Normalization in 30 seconds

- **Batch-Normalization (BN)** is an algorithmic method which makes the training of Deep Neural Networks (DNN) **faster** and **more stable**.

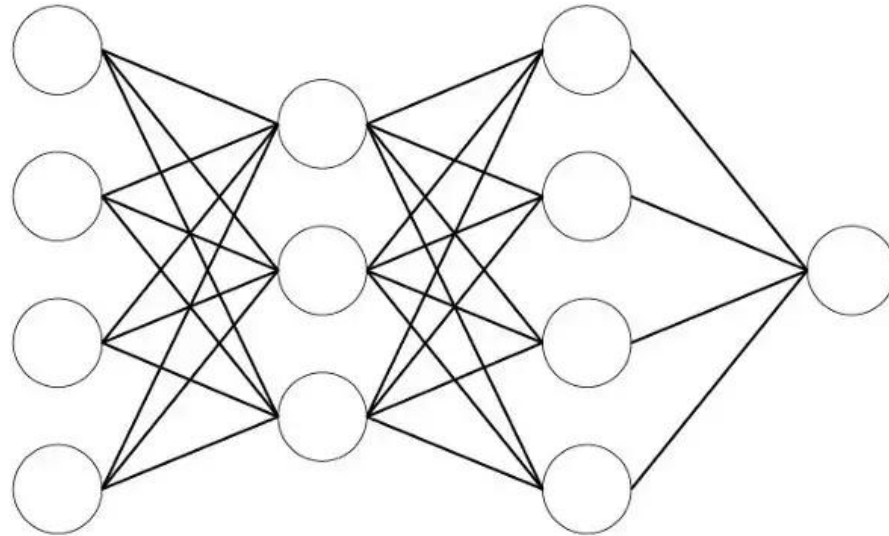
It consists of **normalizing activation vectors from hidden layers** using the mean and variance of the current batch.

This normalization step is applied right before (or right after) the nonlinear function.

Batch Normalization in 30 seconds



- Raw signal
- **High interdependency** between distributions
- **Slow** and **unstable** training



- **Normalized** signal
- **Mitigated interdependency** between distributions
- **Fast** and **stable** training

Batch Normalization in 30 minutes



Batch Normalization in 30 minutes



Covariate Shift

Training Data

Test Data

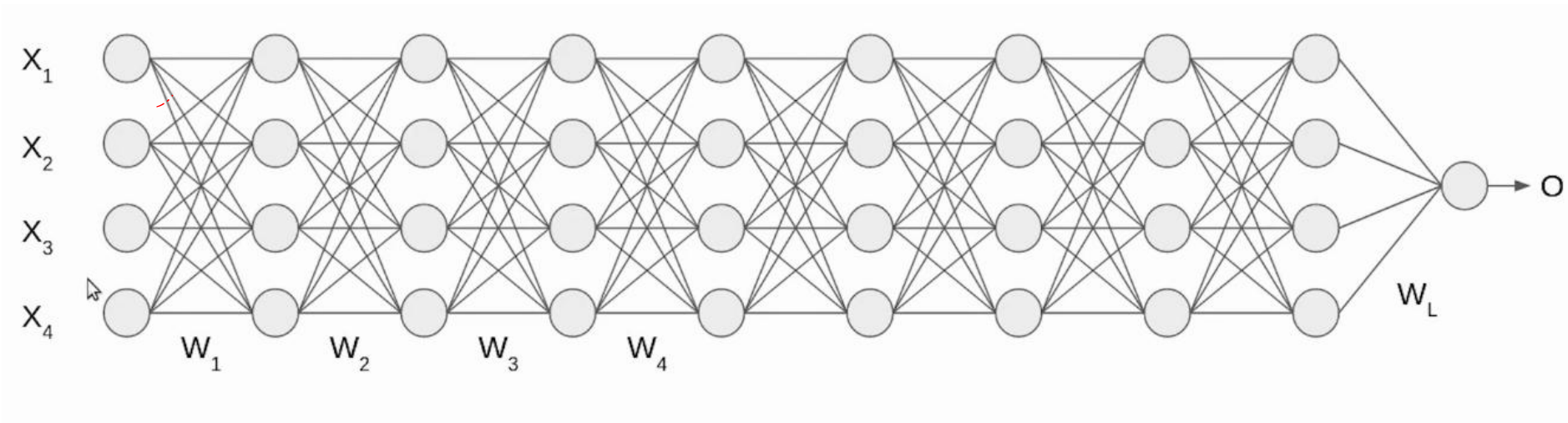
Cat
 $Y = 0$



Dog
 $Y = 1$

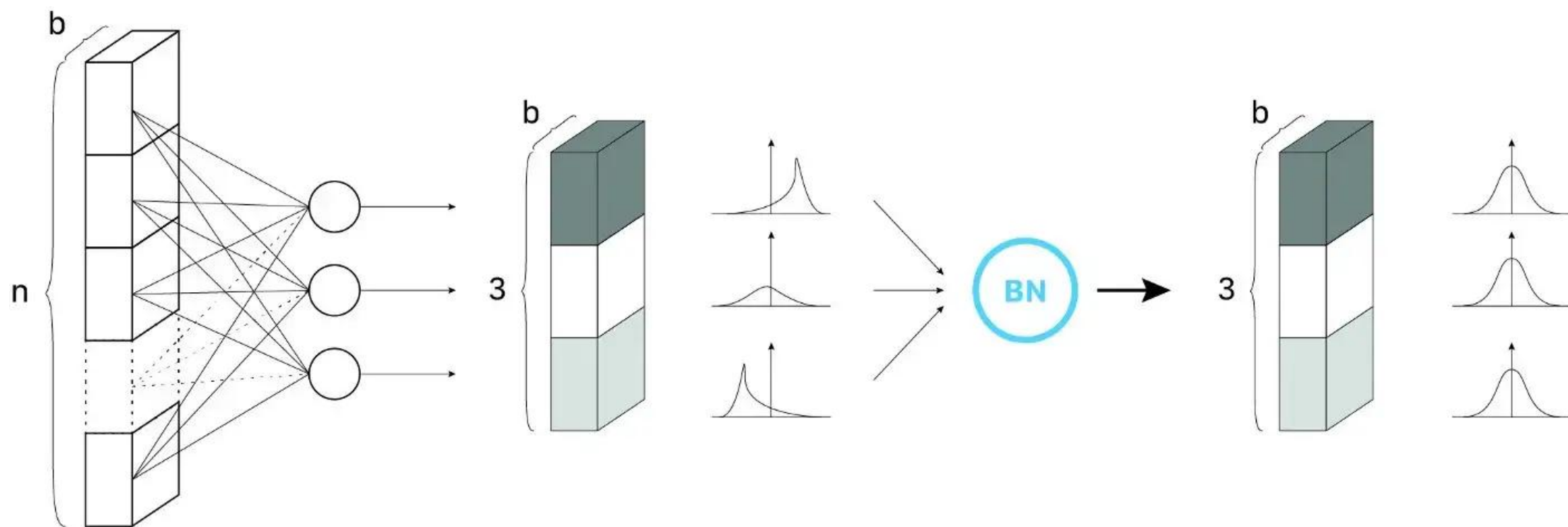


Covariate Shift



***“Internal Covariate Shift** is the change in the distribution of network activations due to the change in network parameters during training.”*

Batch Normalization



Batch Norm – Training Time

First, it zero-centers and normalizes the batch.

$$\mu_B := \frac{1}{N_B} \sum x_B^{(i)}$$

$$\sigma_B^2 := \frac{1}{N_B} \sum (x_B^{(i)} - \mu_B)^2$$

$$\hat{x}_B^{(i)} = \frac{x_B^{(i)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

Then, scales and shifts the batch with two learnable parameters γ, β .

$$y_B^{(i)} = \gamma \hat{x}_B^{(i)} + \beta$$

Batch Norm – Test Time

- To zero-center and normalize the input, we need the average and variance of the whole data.
- Those parameters can be acquired during the training.
- Therefore, we need two more trainable parameters.

$$\mu_D := \frac{1}{N} \sum x^{(i)}$$

$$\sigma_D^2 := \frac{1}{N} \sum (x^{(i)} - \mu_D)^2$$

Batch Norm – Test Time

- The majority of Batch Normalization implementations use an exponential moving average of the layer's input means and standard deviations to estimate these final statistics during training.

$$\begin{aligned}\mu_D &= \alpha\mu_D + (1 - \alpha)\mu_B \\ &= \mu_D - (1 - \alpha)\mu_D + (1 - \alpha)\mu_B \\ &= \mu_D - (1 - \alpha)(\mu_D - \mu_B)\end{aligned}$$

- α is the momentum hyperparameter.
- Based on the equations, older values are lost earlier when momentum is less.
- As a result, the moving average changes more quickly.

Normalization during evaluation mode

Using the same approach makes no sense, because we do not necessarily have an entire batch to predict

We define $(\mu_{\text{pop}}, \sigma_{\text{pop}})$, which are respectively the estimated mean and standard deviation of the targeted population

Batch Normalization Effects



Batch Normalization Effects

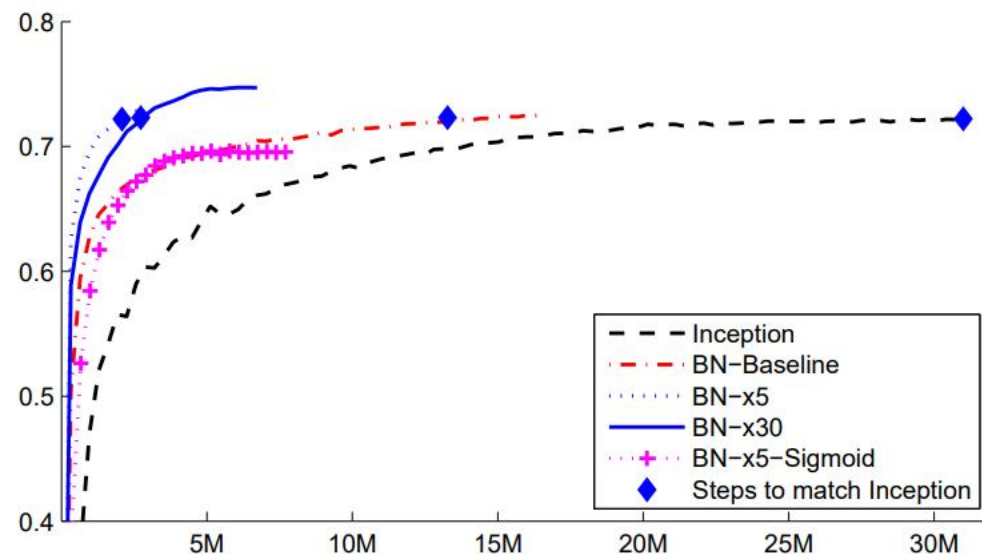


Figure 2: *Single crop validation accuracy of Inception and its batch-normalized variants, vs. the number of training steps.*

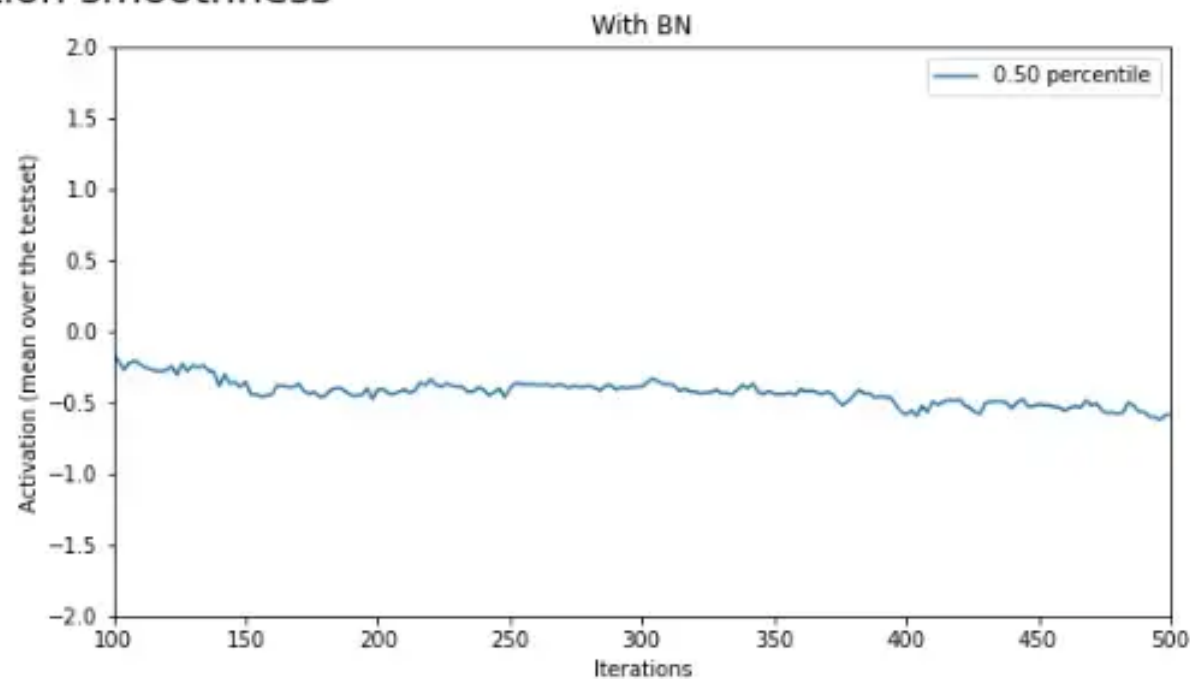
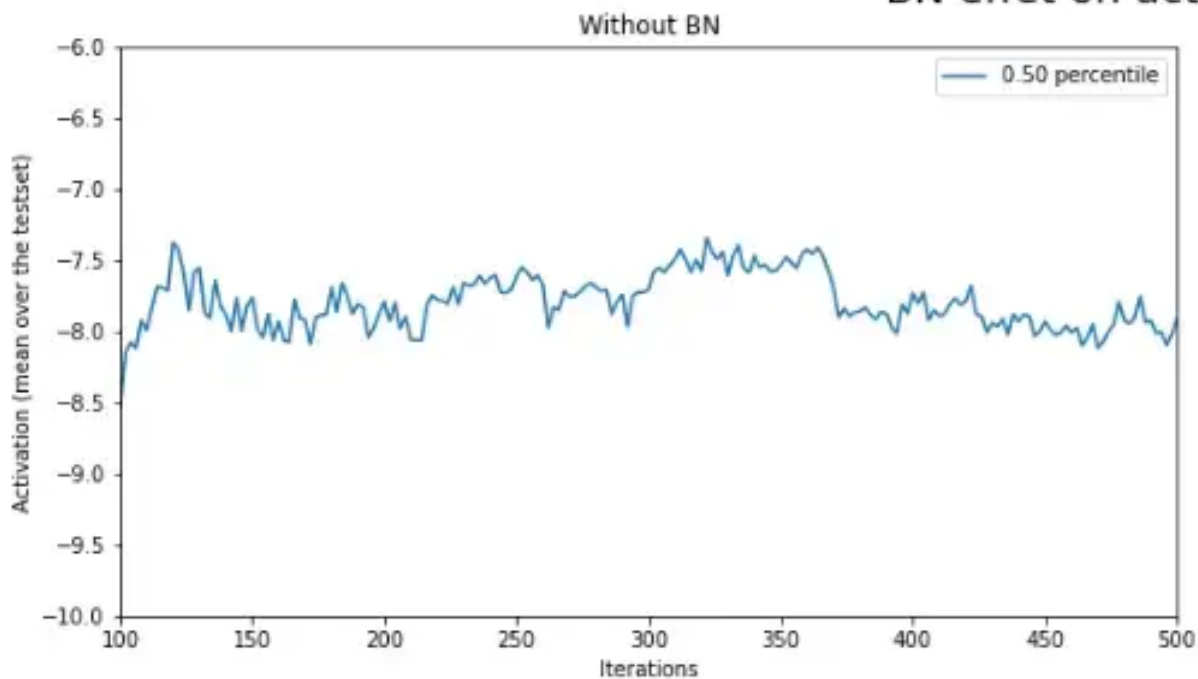
- **BN-Baseline:** Same learning rate as Inception.
- **BN-x5:** Initial learning rate of 0.0075 (5 times Inception's learning rate).
- **BN-x30:** Initial learning rate 0.045 (30 times that of Inception).
- **BN-x5-Sigmoid:** Uses Sigmoid activation function (non-linearity) instead of ReLU.

Model	Steps to 72.2%	Max accuracy
Inception	$31.0 \cdot 10^6$	72.2%
BN-Baseline	$13.3 \cdot 10^6$	72.7%
BN-x5	$2.1 \cdot 10^6$	73.0%
BN-x30	$2.7 \cdot 10^6$	74.8%
BN-x5-Sigmoid		69.8%

Figure 3: *For Inception and the batch-normalized variants, the number of training steps required to reach the maximum accuracy of Inception (72.2%), and the maximum accuracy achieved by the network.*

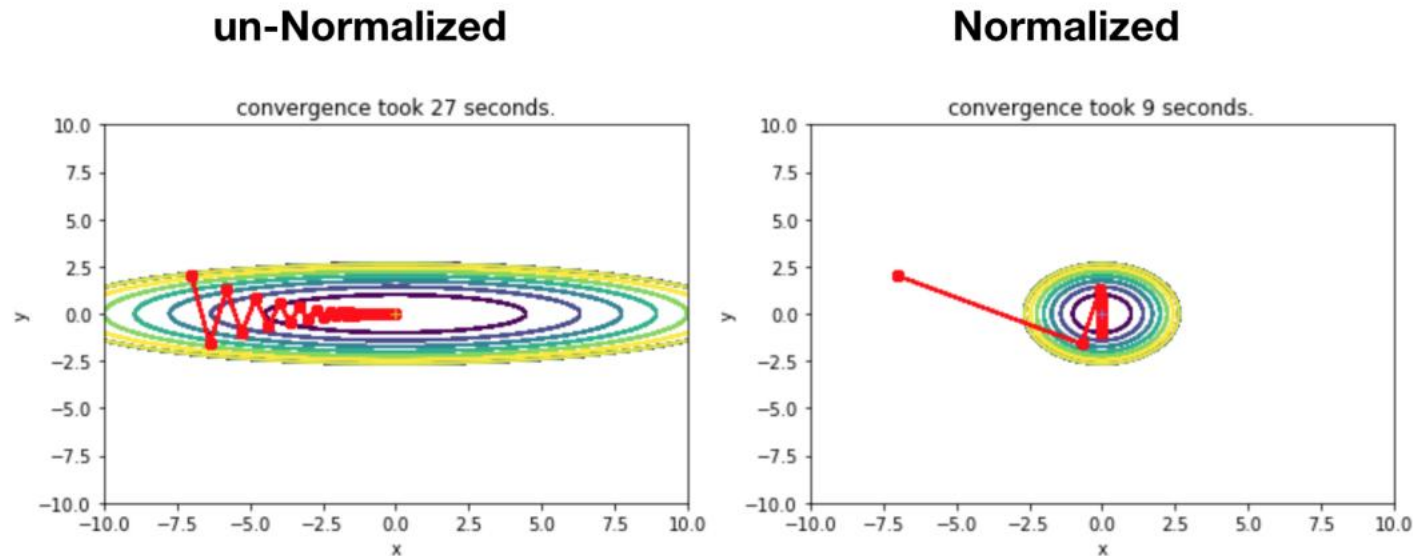
Batch Normalization Effects

BN effect on activation smoothness



Batch Normalization Effects

Normalizing the data improves the convergence speed by a considerable amount.

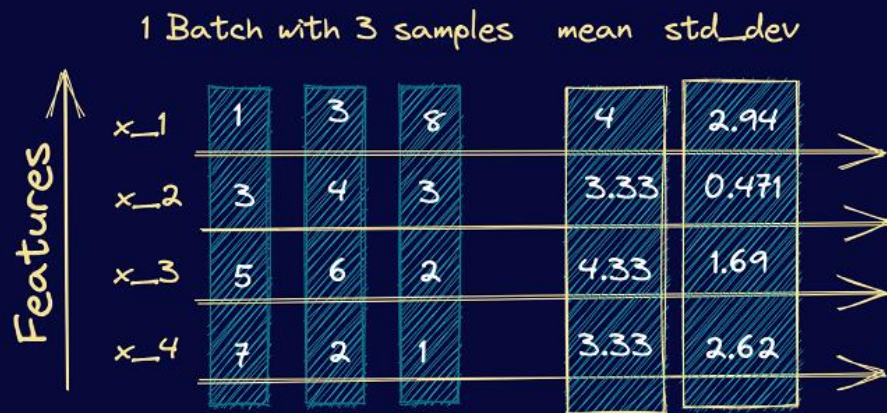


Limitations of Batch Normalization

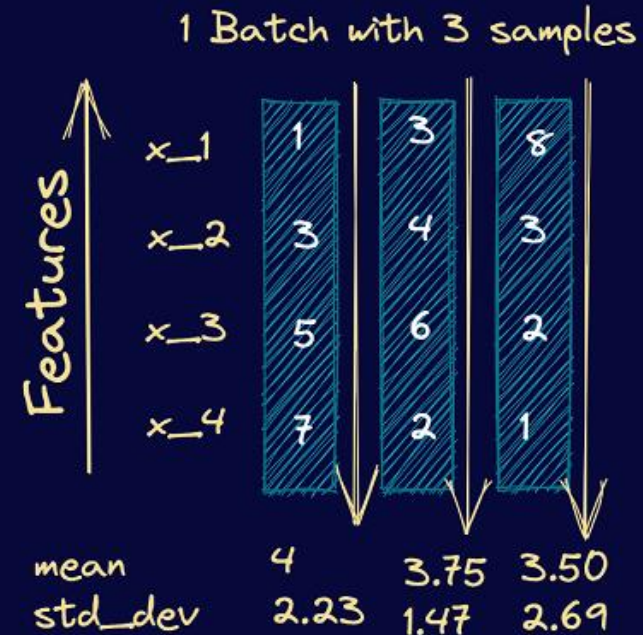
- When the batch size is small :
 - the sample mean and sample standard deviation are not representative enough of the actual distribution
 - the network cannot learn anything meaningful.
- Batch normalization is less suited for sequence models.
 - in sequence models, we may have sequences of potentially different lengths and smaller batch sizes corresponding to longer sequences.

Layer Normalization

Batch Normalization vs Layer Normalization



Normalization across mini-batch,
independently for each feature



Normalization across features,
independently for each sample

Batch Norm Summary

Pros

- ▶ Vanishing/Exploding gradient problem is reduced by a considerable amount.
- ▶ You can use even saturating activation functions.
- ▶ The network is much less sensitive to the initial weight.
- ▶ We're able to use larger learning rates, which speeds up the training.
- ▶ It also acts as a regularizer.
 - There is no need for other regularizer techniques.

Batch Norm Summary

Q : Dose Order matter?

Cons

- ▶ It increases model parameters and prediction latency.

Solution: during the test time, we can mix the BatchNorm layer with its previous layer to hold the prediction latency.

$$\begin{aligned} x'^{(i)} &= Wx^{(i)} + b \\ y^{(i)} &= \frac{x'^{(i)} - \mu_D}{\sqrt{\sigma_D^2 + \epsilon}} + \beta \end{aligned} \quad \Rightarrow \quad \begin{aligned} y^{(i)} &= W'x^{(i)} + b' \\ W' &:= \frac{1}{\sqrt{\sigma_D^2 + \epsilon}} W \\ b' &:= \beta + \frac{b - \mu_D}{\sqrt{\sigma_D^2 + \epsilon}} \end{aligned}$$

Where to apply batch normalization?



- **After ReLU or before?**

- The original batch-norm paper prescribes using the batch-norm before ReLU activation. But there is evidence that it's probably better to use batchnorm *after* the activation.

<https://github.com/keras-team/keras/issues/187966878-tnemmoceussi#1802>

- **After Pooling or before?**

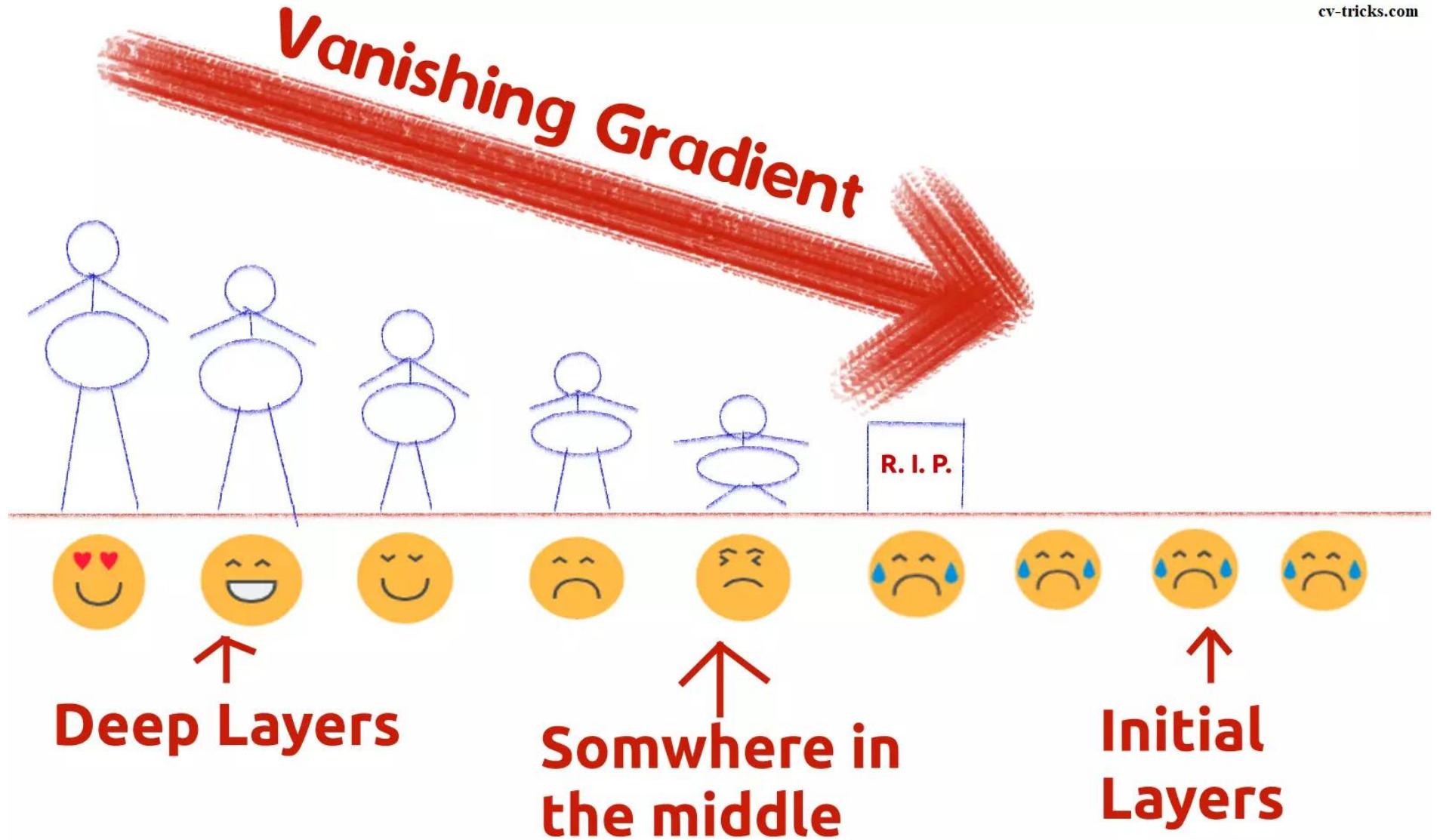
- After pooling probably a little bit faster

The Deeper, the Better



Vanishing Gradient

cv-tricks.com



ResNet

Deep Residual Learning for Image Recognition

Kaiming He

Xiangyu Zhang

Shaoqing Ren

Jian Sun

Microsoft Research



Kaiming He

Research Scientist, Facebook AI Research (FAIR)

Verified email at fb.com - [Homepage](#)

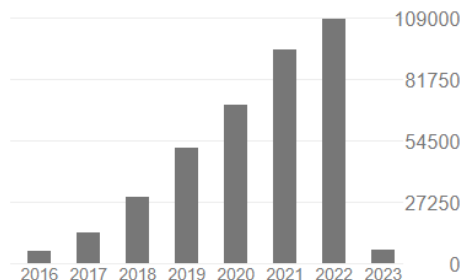
[Computer Vision](#) [Machine Learning](#)

FOLLOW

[GET MY OWN PROFILE](#)

TITLE	CITED BY	YEAR
Deep Residual Learning for Image Recognition K He, X Zhang, S Ren, J Sun Computer Vision and Pattern Recognition (CVPR), 2016	152733	2016
Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks S Ren, K He, R Girshick, J Sun Neural Information Processing Systems (NIPS), 2015	52799	2015
Mask R-CNN K He, G Gkioxari, P Dollár, R Girshick International Conference on Computer Vision (ICCV), 2017	24566	2017
Focal Loss for Dense Object Detection TY Lin, P Goyal, R Girshick, K He, P Dollár International Conference on Computer Vision (ICCV), 2017	18317	2017

Cited by	VIEW ALL	
	All	Since 2018
Citations	388879	362259
h-index	64	62
i10-index	71	70



ResNet

Deep Residual Learning for Image Recognition

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun
Microsoft Research



Kaiming He

Associate Professor, EECS, [MIT](#)
Verified email at mit.edu - [Homepage](#)
[Computer Vision](#) [Machine Learning](#)

 FOLLOW

TITLE	CITED BY	YEAR
Deep Residual Learning for Image Recognition K He, X Zhang, S Ren, J Sun Computer Vision and Pattern Recognition (CVPR), 2016	287279	2016
Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks S Ren, K He, R Girshick, J Sun Neural Information Processing Systems (NIPS), 2015	93659	2015
Mask R-CNN K He, G Gkioxari, P Dollár, R Girshick International Conference on Computer Vision (ICCV), 2017	46253	2017
Focal Loss for Dense Object Detection TY Lin, P Goyal, R Girshick, K He, P Dollár International Conference on Computer Vision (ICCV), 2017	40568	2017

ResNet

Deep Residual Learning for Image Recognition

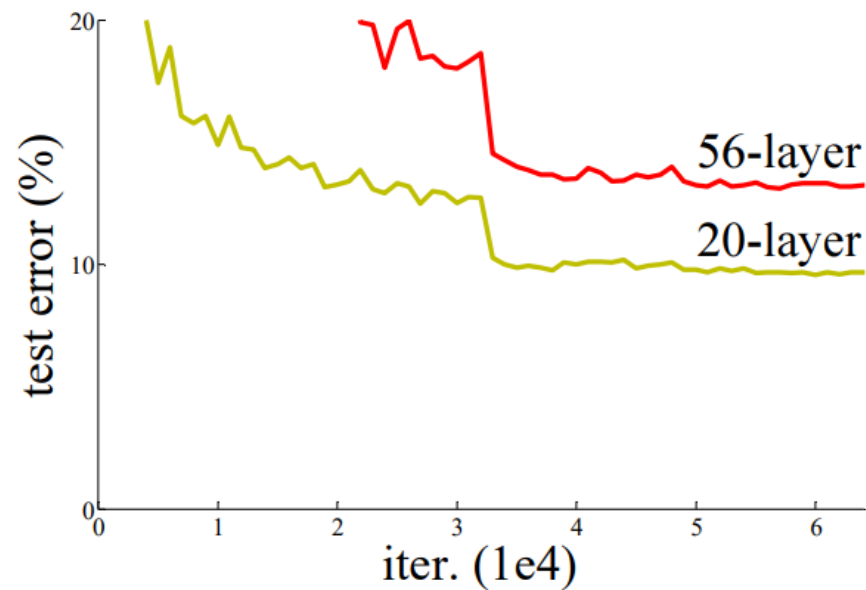
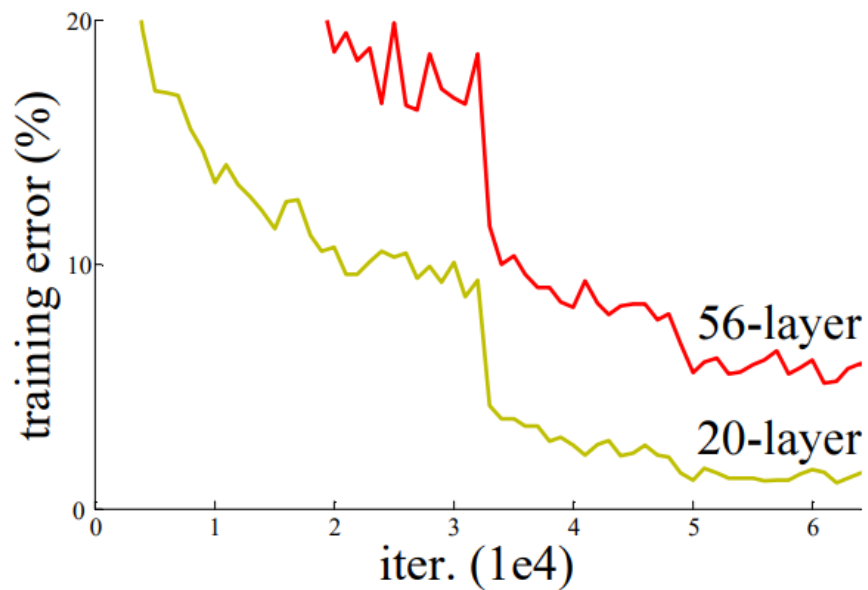
Kaiming He

Xiangyu Zhang

Shaoqing Ren

Jian Sun

Microsoft Research



Overfit?

Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network

Core Idea

We want the deep network to perform at least as good as the shallow network and not degrade the performance

Is it possible? 🤔



Shallow Net



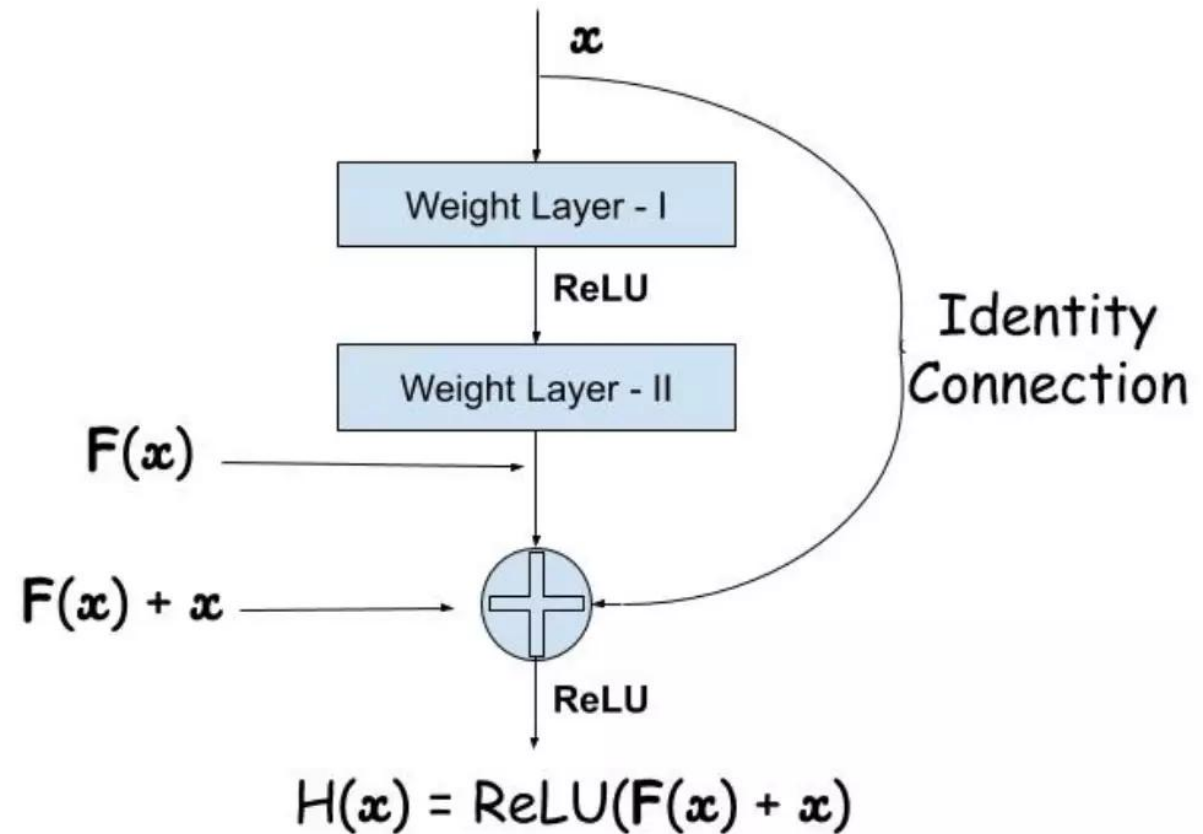
Identity mapping

Deep Net

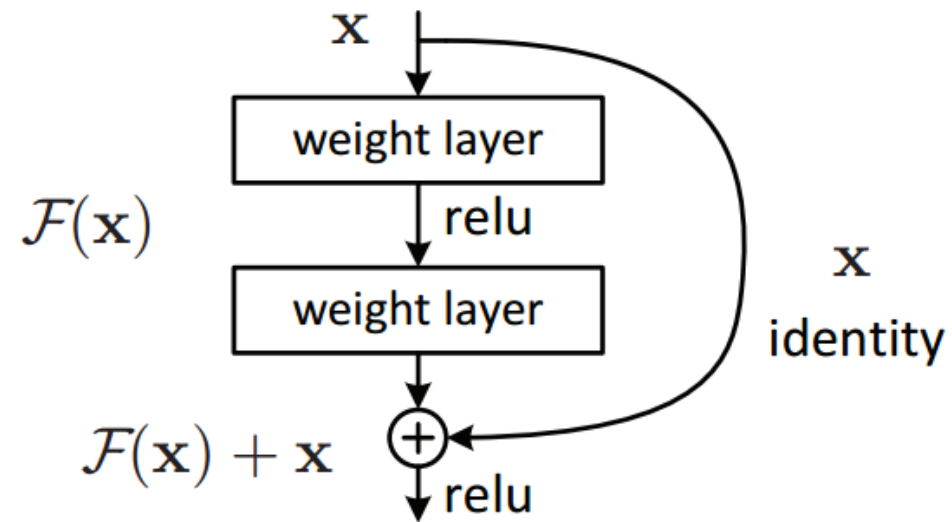
Skip Connection and Residual Block

Skip connections help is by allowing the model to learn the identity functions which ensures that the higher layer will perform at least as good as the lower layer.

thus we expect ResNet to perform equally or better than the plain deep neural networks



why we call it residual ?



Layers in a traditional network are learning the **true output ($H(x)$)**, whereas the layers in a residual network are learning **the residual** ($R(x)$ or $F(x)$). Hence, the name: ***Residual Block***

Residual Block Effect

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	25.03

Table 2. Top-1 error (% , 10-crop testing) on ImageNet validation. Here the ResNets have no extra parameter compared to their plain counterparts. Fig. 4 shows the training procedures.

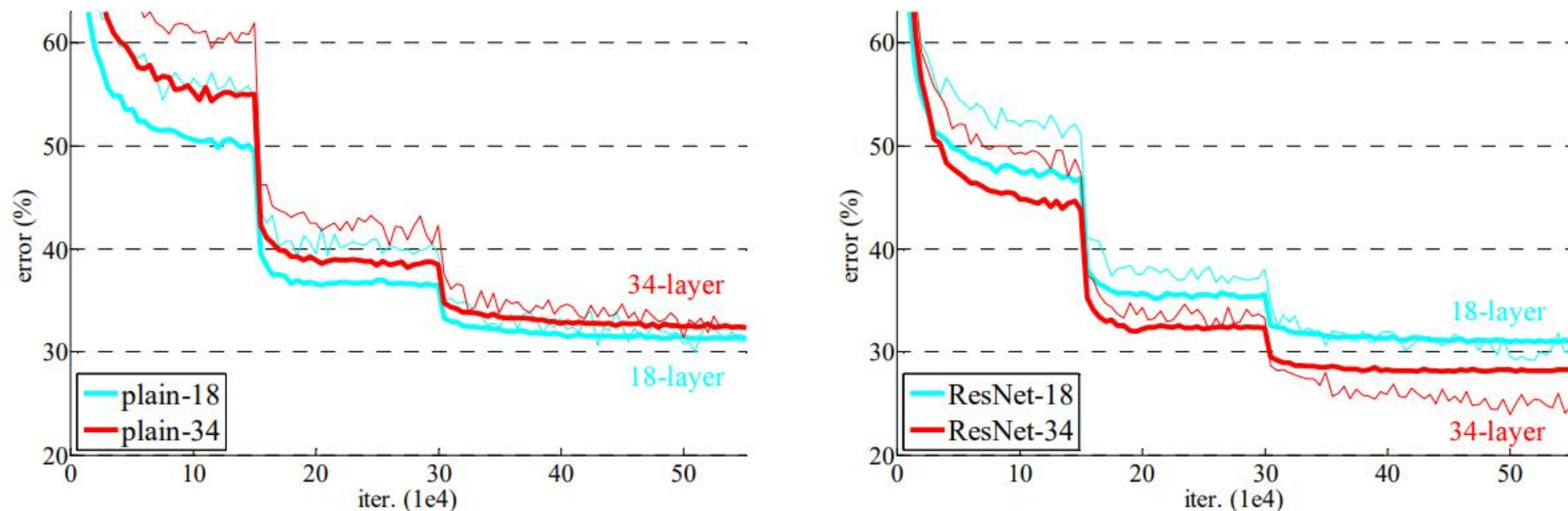
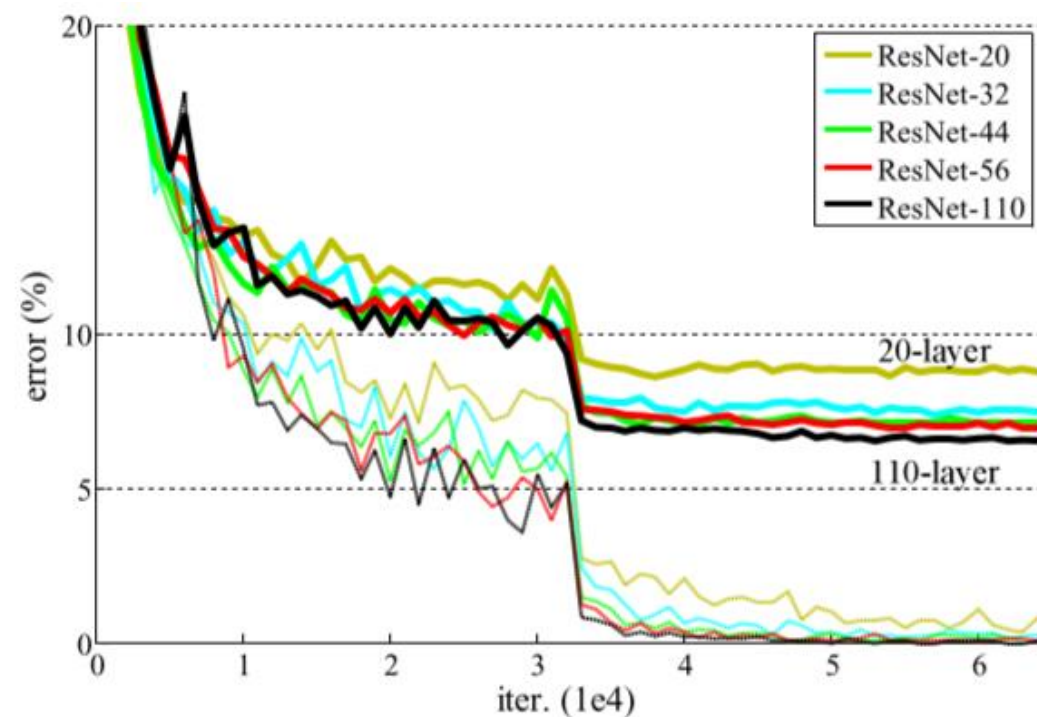
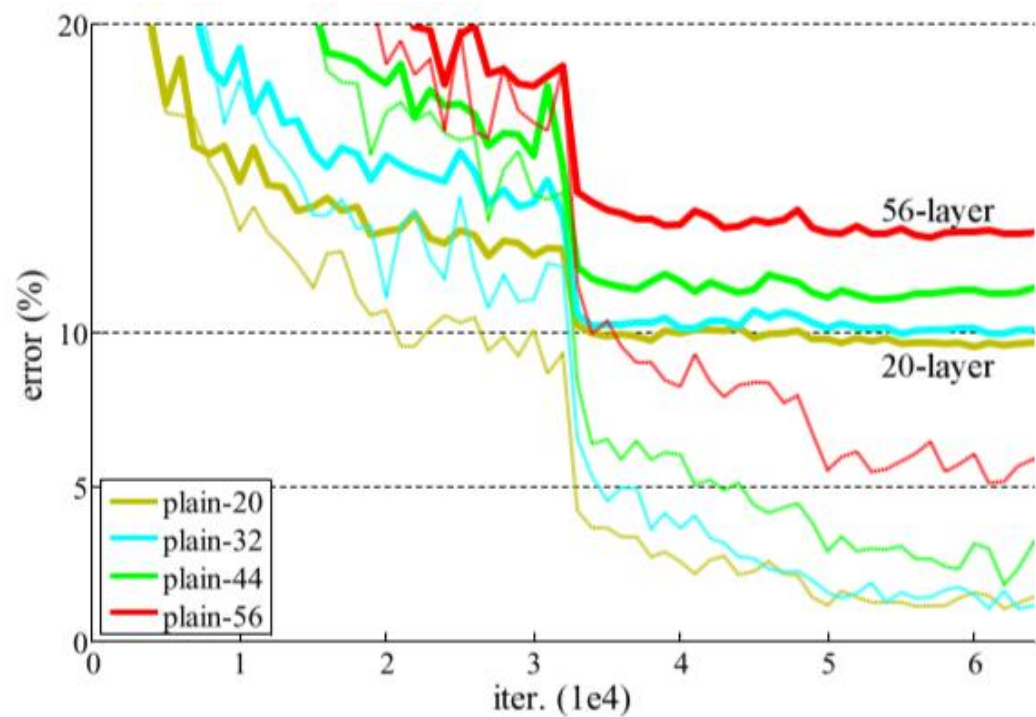


Figure 4. Training on **ImageNet**. Thin curves denote training error, and bold curves denote validation error of the center crops. Left: plain networks of 18 and 34 layers. Right: ResNets of 18 and 34 layers. In this plot, the residual networks have no extra parameter compared to their plain counterparts.

Residual Block Effect



ResNet Architecture

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7 , 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Example network architectures for ImageNet.

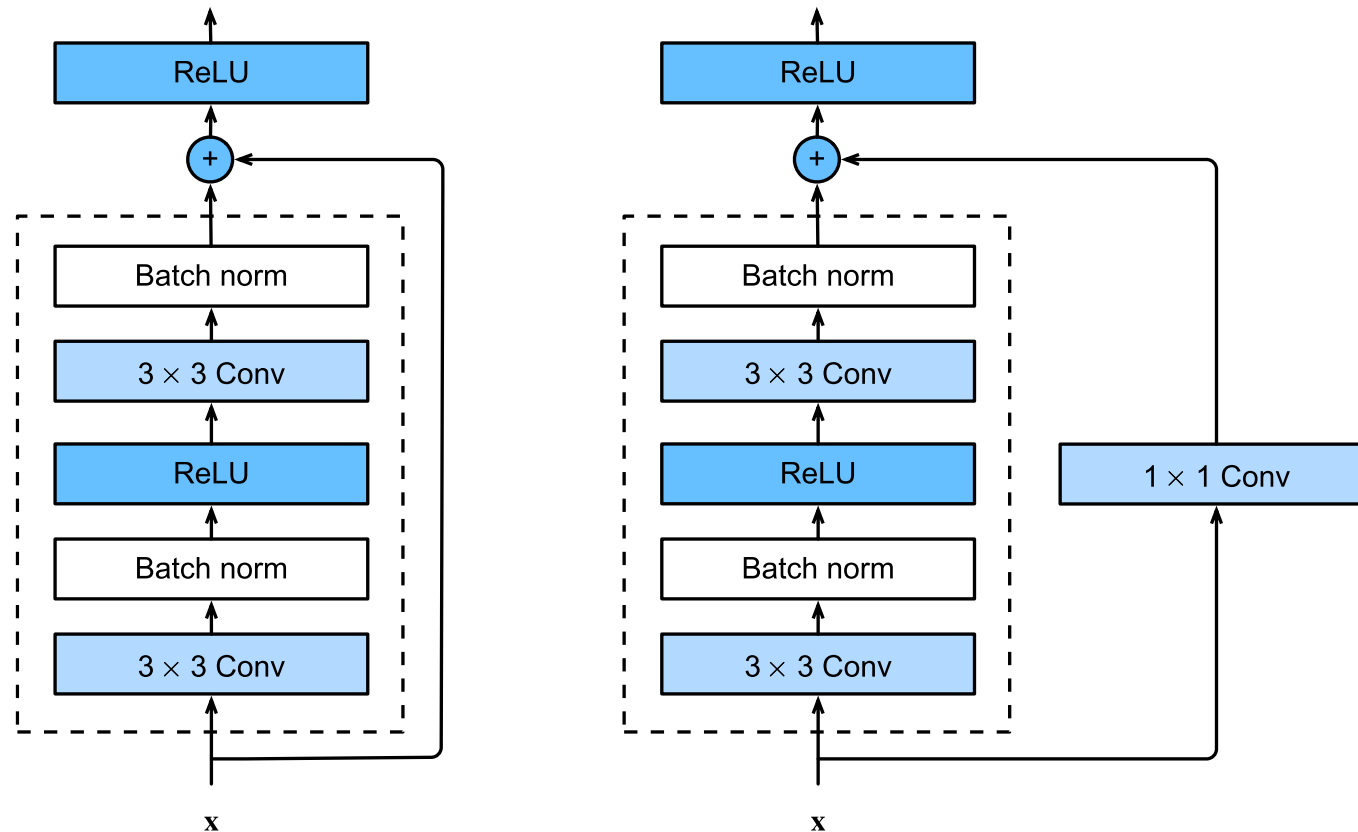
Left: the VGG-19 model (19.6 billion FLOPs).

Middle: a plain network with 34 parameter layers (3.6 billion FLOPs)

Right: a residual network with 34 parameter layers (3.6 billion FLOPs)



Input/output dimension problem



Bottleneck Design

- 1x1 conv faster than 3x3 conv

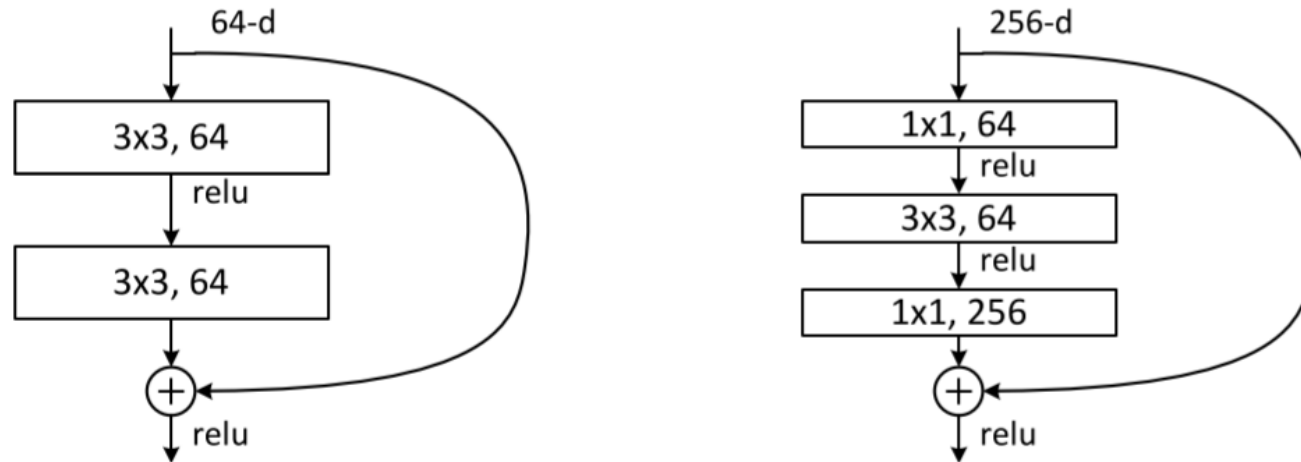


Figure 5. A deeper residual function $\mathcal{F}$ for ImageNet. Left: a building block (on 56×56 feature maps) as in Fig. 3 for ResNet-34. Right: a “bottleneck” building block for ResNet-50/101/152.

Summary - Resnet

- 1.Speed up the training
- 2.Improve the performance of classification
- 3.Release the power of deeper neural network

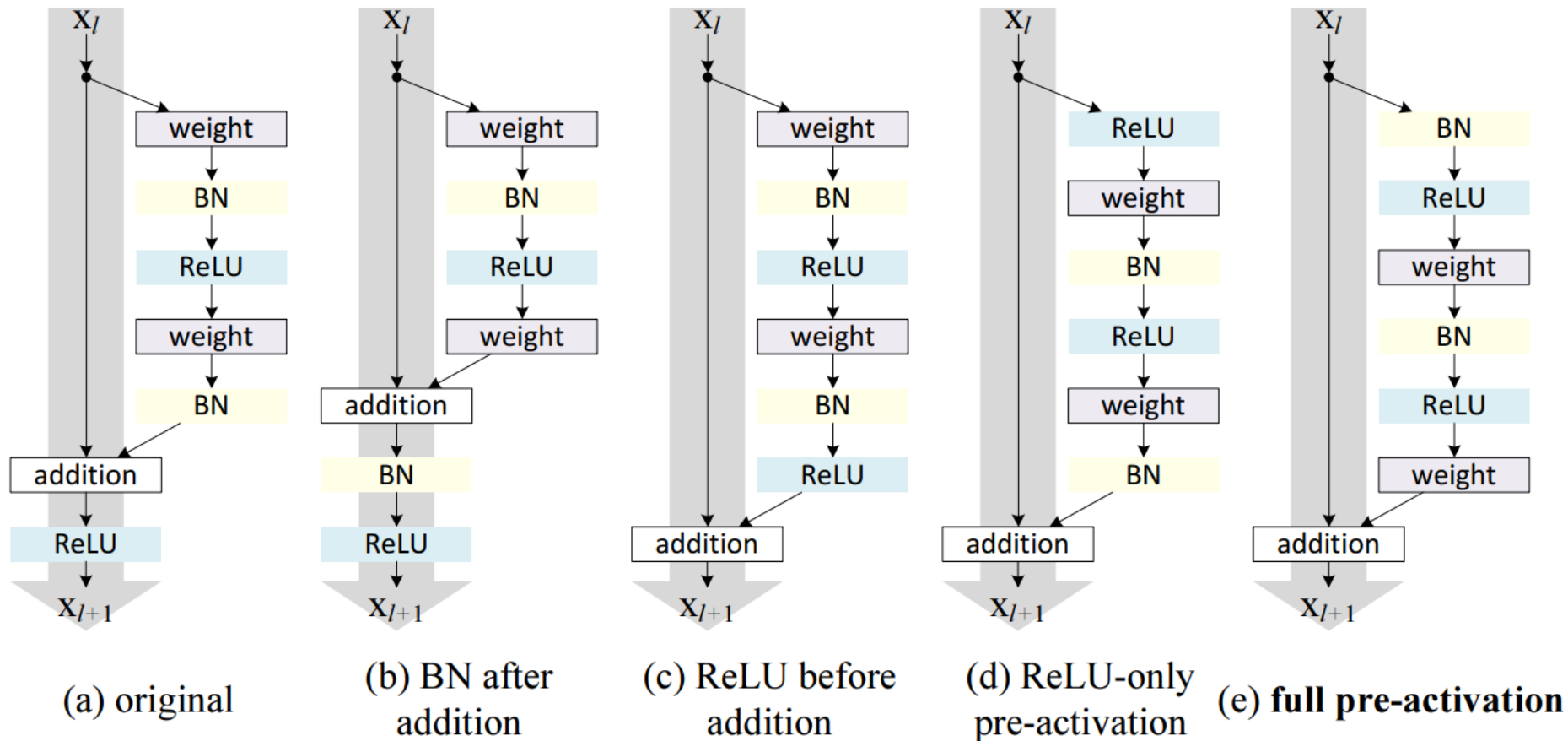
Types of Residual Block

Identity Mappings in Deep Residual Networks

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun

Microsoft Research

<https://arxiv.org/pdf/fdp.1603.05027>



Wide ResNet

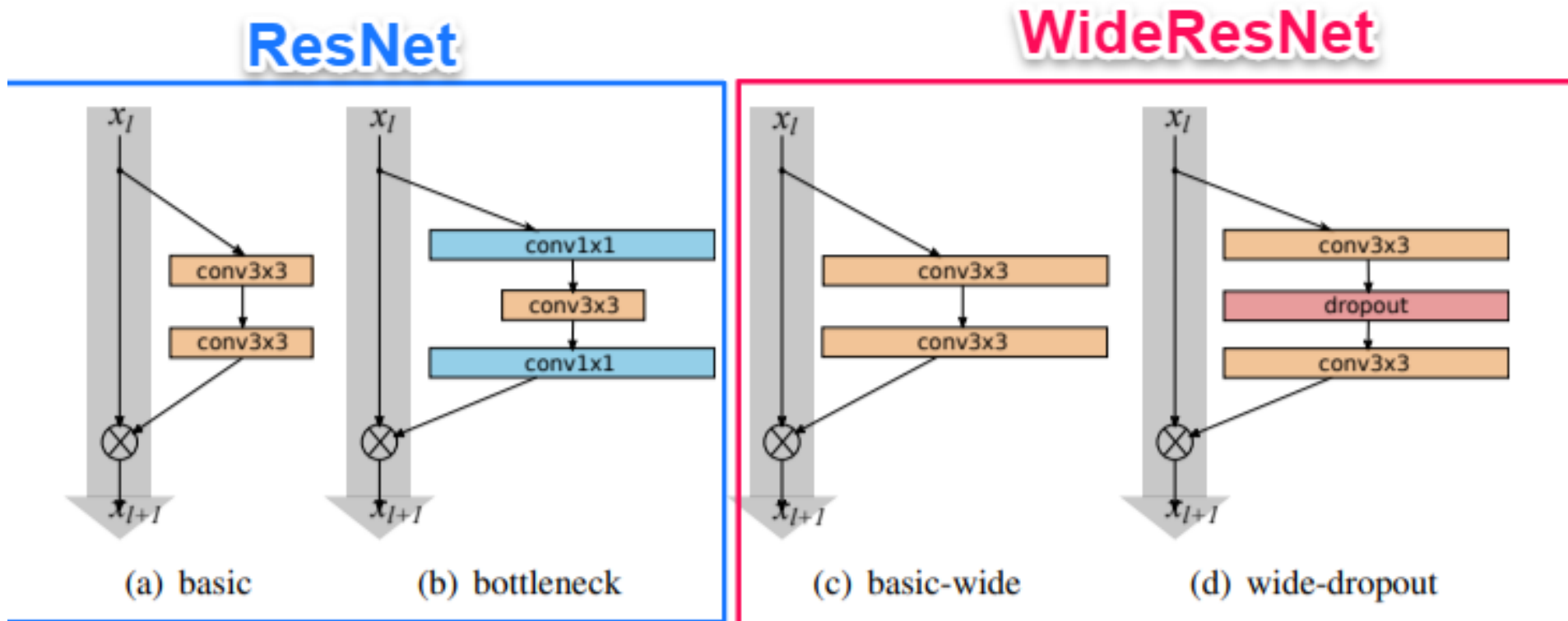
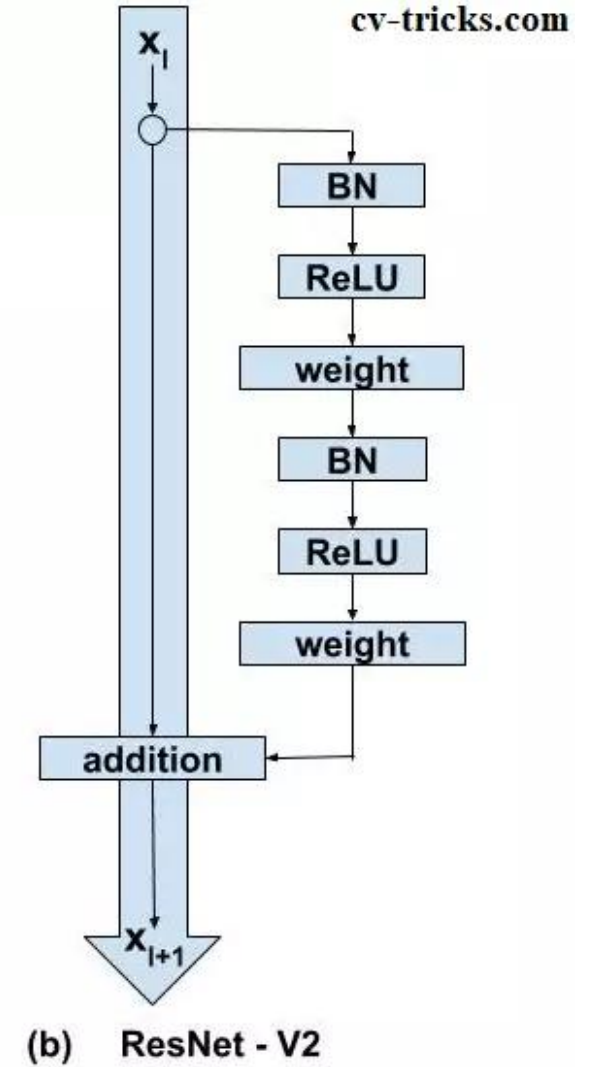
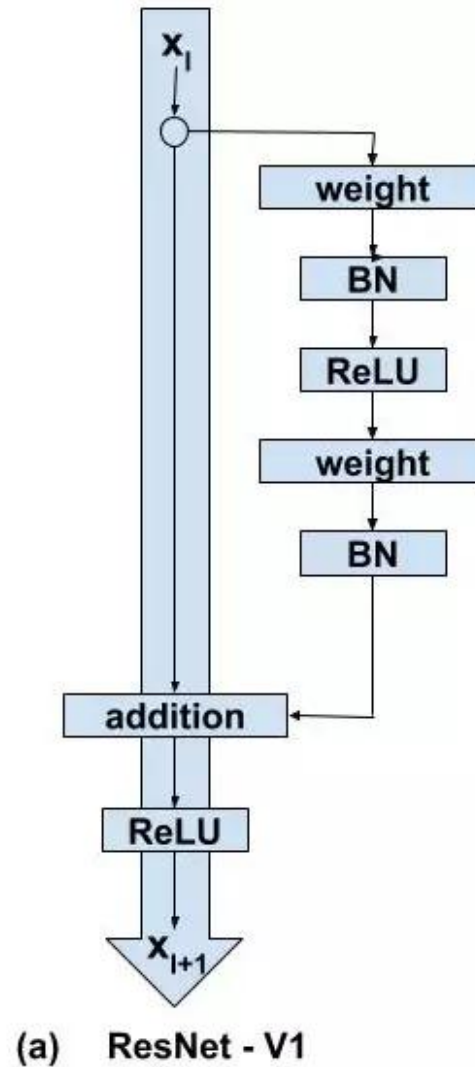


Figure 1: Various residual blocks used in the paper. Batch normalization and ReLU precede each convolution (omitted for clarity)

ResNet v2

The primary difference between ResNetV2 and the original (V1) is that V2 uses batch normalization before each weight layer.



ResNext

- Create a multi-path model with more general representation

Inspired by Inception, ResNeXt integrates the multi-path into ResNet.

- the multi-path in Inception is well designed by doing huge number of experiments.

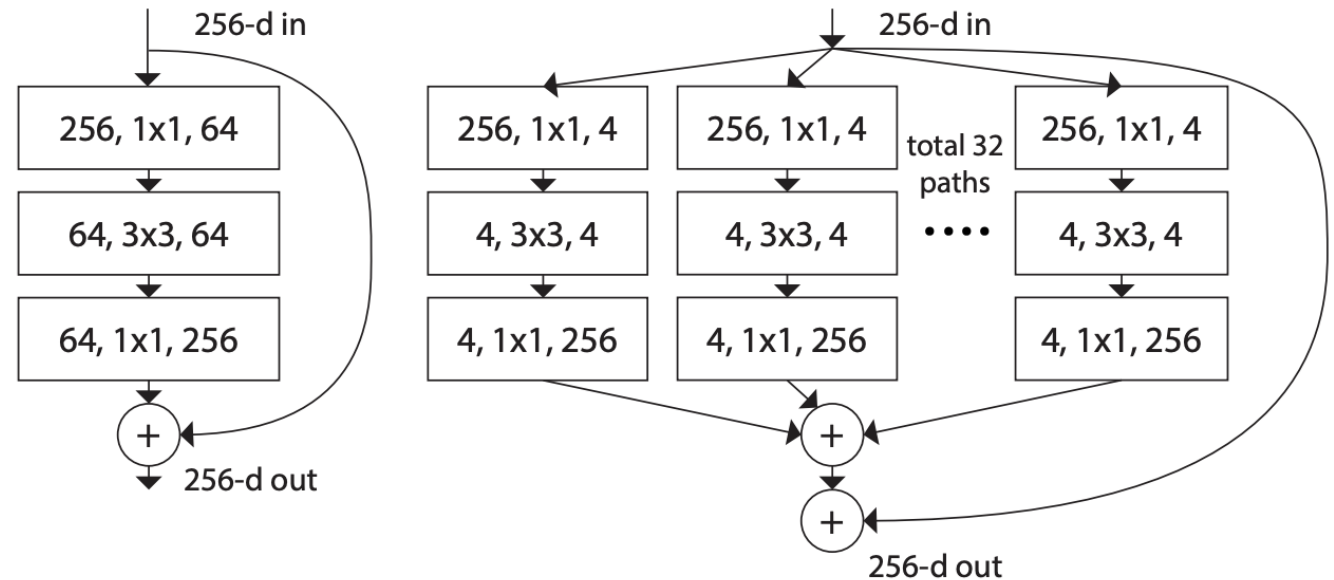


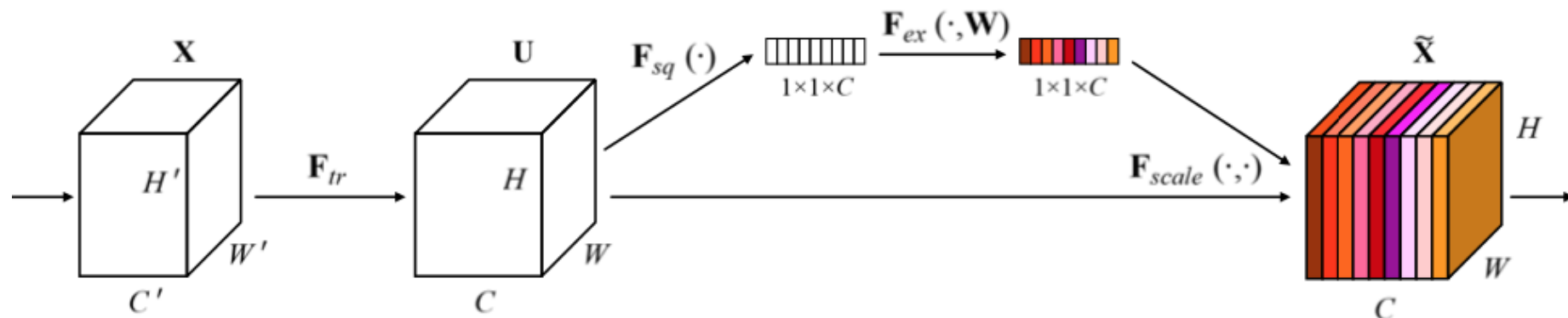
Figure 1. **Left:** A block of ResNet [14]. **Right:** A block of ResNeXt with cardinality = 32, with roughly the same complexity. A layer is shown as (# in channels, filter size, # out channels).

SENet

ResNet and ResNeXt focus on the operation on the spatial information. SENet develops channel-wise operation by introducing the attention mechanism. The authors also names it as recalibration. With the weights generated from the attention mechanism or recalibration, SENet rescale the value in different channel

The attention mechanism of SENet includes 3 parts:

- 1.Squeeze: Global average pooling is used to get global information embedding
- 2.Excitation: Encode and decode to get adaptive recalibration
- 3.Scale by channel-wise multiplication of excitation and input



SKNet

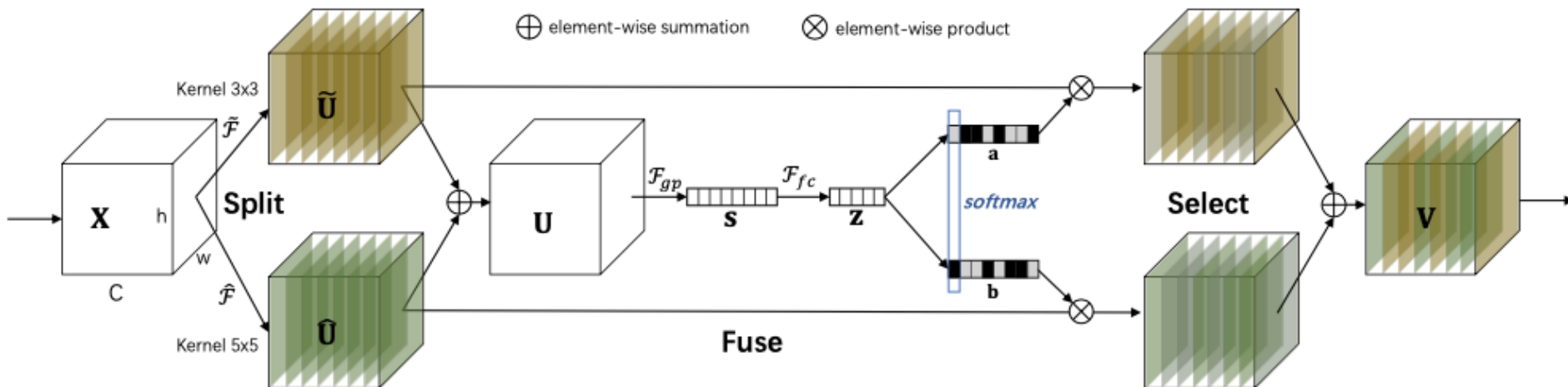
Selective Kernel Networks (SKNet)

is designed to automatically choose the size of receptive field. The input will be processed by several kernels with different size (e.g. 3x3, 5x5)

1.split: Perform convolutions with kernels of different size, (n kernels)

2.fuse: Element-wise adding of results processed by different kernels

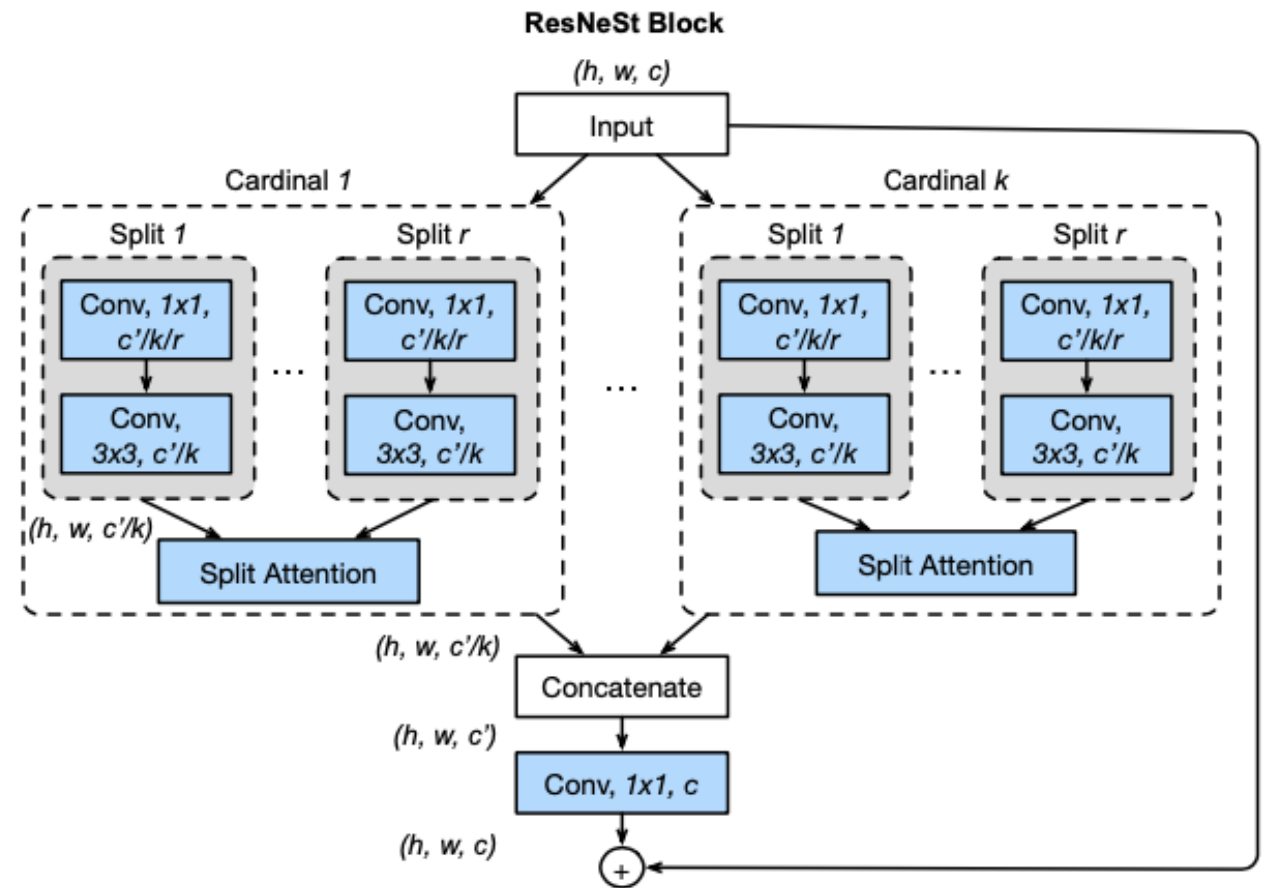
3.select: The fused information is firstly processed by global embedding and dimension reduction. The output is processed by 2 different matrix (which is learned during training) to generate n softmax value to scale the information generated by step 1, then do element-wise summation of the rescaled information.



ResNeSt

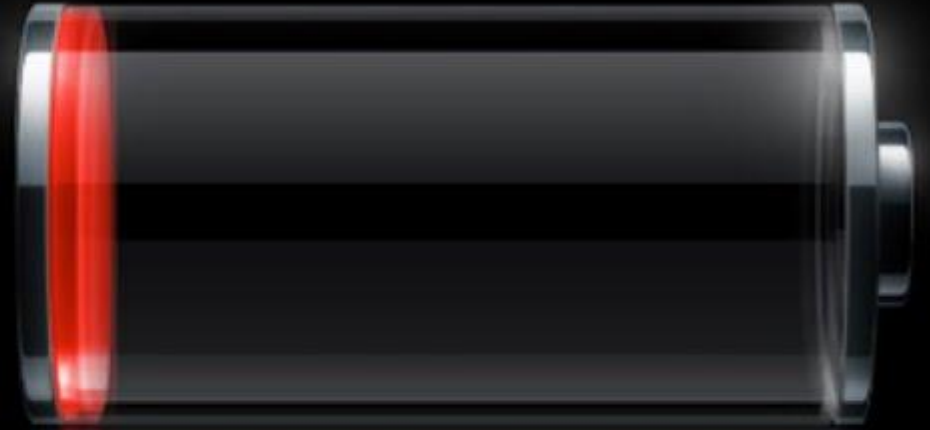
ResNeSt is said to be an approach to integrate all the module together.

ResNeSt firstly performs as ResNeXt to apply multi-path of the input, then in each cardinal (path) the split attention is used. In the SKNet, the split is done by apply convolution with different kernel size and the purpose is to select best kernel size channel-wise. However, ResNeSt do convolution with the kernels with same size. The authors also mention it's branch-selection. I think it's more like to generate several model and ensemble them together with split attention. There is some differences here comparing to SKNet



*Your neural network is only as good as
the data you feed it.*

Low
Data



Data, Model, Hardware

Data Hungry

- Semi-supervised learning
- Unsupervised learning
- Few-shot learning
- Generative Models
- Transfer learning
- Data Augmentation



What people think AI is about



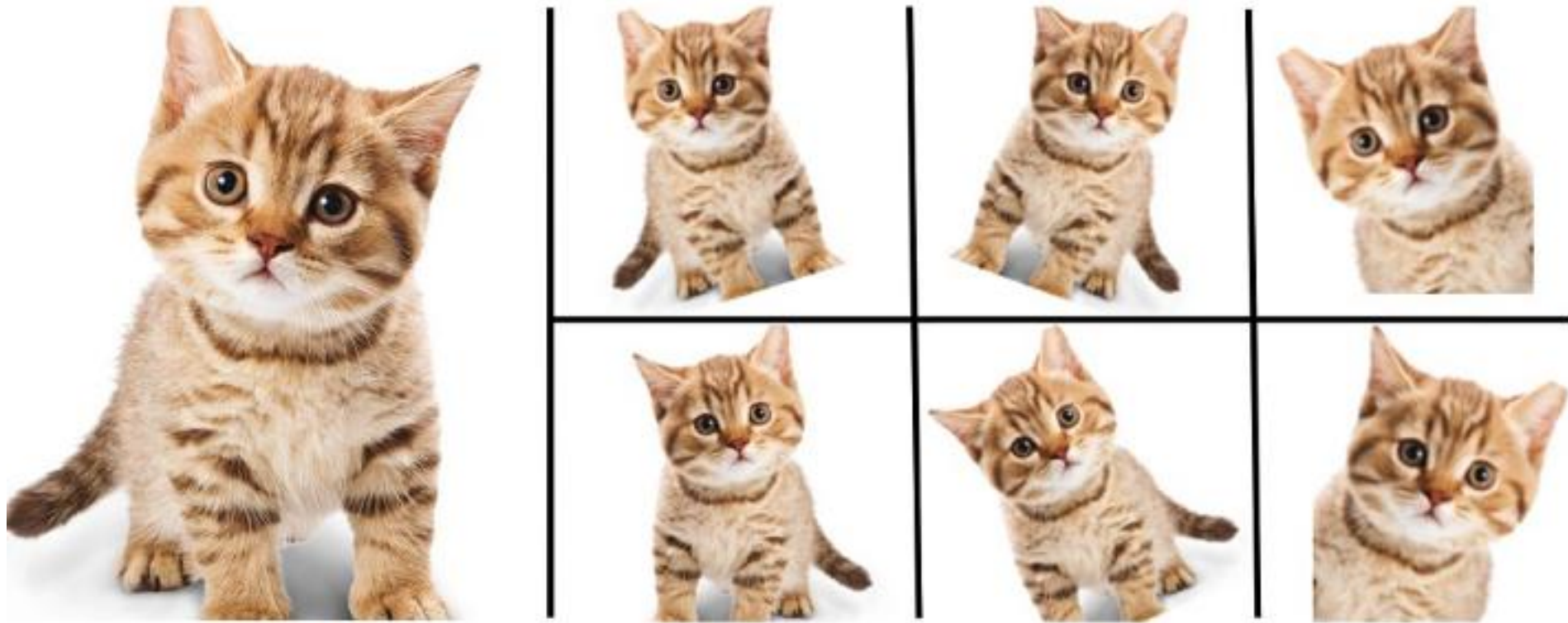
The reality

Case Study

Real data + Generated Data + Augmentation

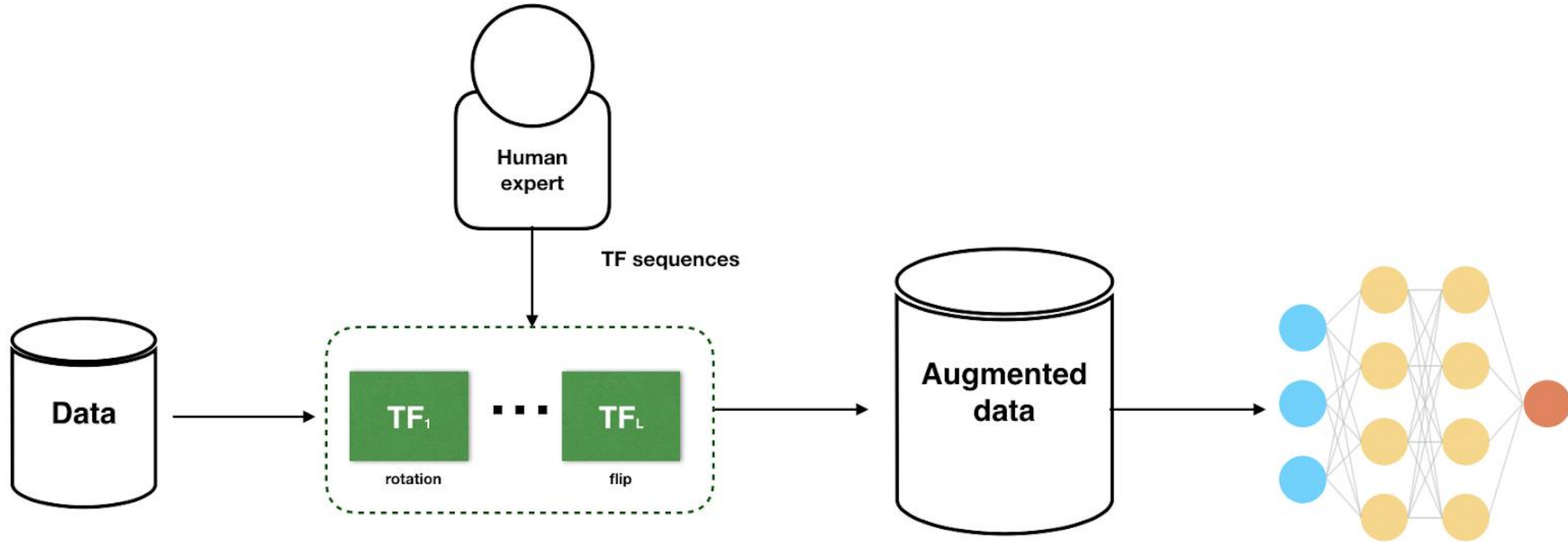


Data Augmentation



Enlarge your Dataset

Data Augmentation



Data Augmentation Types

Offline augmentation

- preferred for **Smaller datasets**

Online augmentation real-time Augmentation/ augmentation on the fly

- preferred for **larger datasets**

Auto Augment

<https://arxiv.org/pdf/fdp.1805.09501>

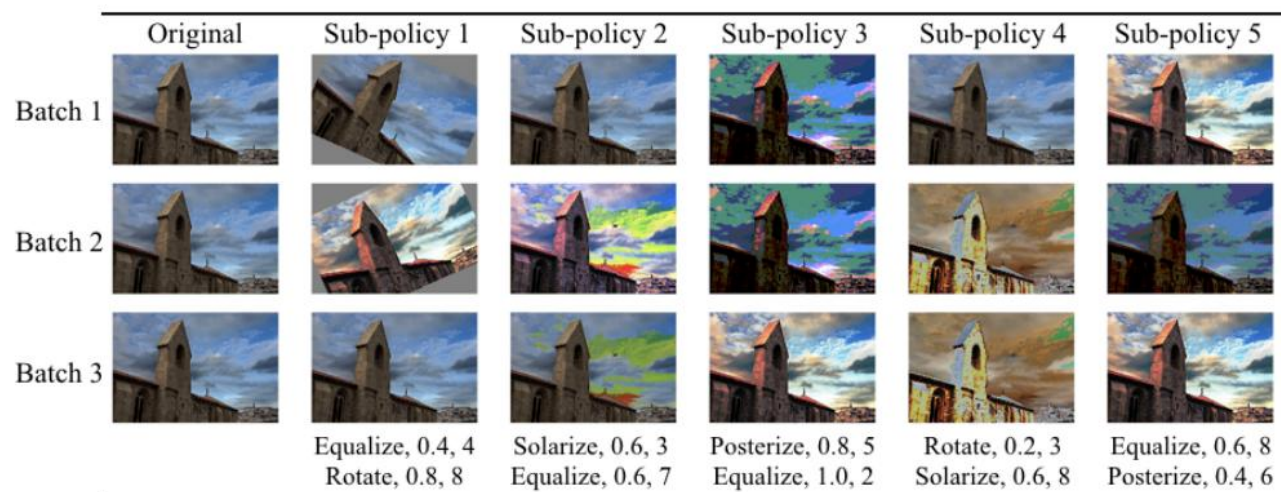


Figure 3. One of the successful policies on ImageNet. As described in the text, most of the policies found on ImageNet used color-based transformations.

Method	Baseline	Augmented	Improvement Δ
LSTM [47]	7.7	6.0	1.6
MF [47]	7.7	5.6	2.1
AutoAugment (ResNet-32)	7.7	4.5	3.2
AutoAugment (ResNet-56)	6.6	3.6	3.0

Table 5. The test set error rates (%) on CIFAR-10 with different approaches for automated data augmentation. The MF and LSTM results are taken from [47], and they are for a ResNet-56.



Do more with less data

- **CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features**
- https://keras.io/api/keras_cv/layers/
- <https://demo.albumentations.ai/>